

MovieLens

Προετοιμασία προφορικής εξέτασης

168 ερωτήσεις με απαντήσεις
16 ασκήσεις DevTools · 3 mock exams

HTML / CSS / JavaScript
HTTP / FastAPI / SQLite
Αλγόριθμος recommendations και υπόλοιπη θεωρία διαλέξεων

Ελληνικές εξηγήσεις με English technical terms.
Βασισμένο στις διαλέξεις, στις εκφωνήσεις Άνοιξης/Ιουνίου 2026 και στον κώδικα της εργασίας.

10 Σεπτεμβρίου 2026

Οι προτεινόμενες ερωτήσεις είναι εξάσκηση από την ύλη, όχι εγγυημένη λίστα μελλοντικών θεμάτων.

Περιεχόμενα

Τα κεφάλαια είναι συνδεδεμένα. Χρησιμοποίησε επίσης τα PDF bookmarks για τις αριθμημένες ερωτήσεις.

MovieLens: προετοιμασία προφορικής εξέτασης	3
1. Αρχιτεκτονική και απαιτήσεις	5
2. HTTP, URLs, CORS και browser networking	8
3. JavaScript: async, τύποι, scope και functions	12
4. HTML, CSS, DOM και Inspect Element	17
5. FastAPI, Pydantic και ο κύκλος του request	20
6. SQLite, SQL και διαχείριση δεδομένων	23
7. Recommendations: κατανόηση και αριθμητικό παράδειγμα	25
8. Υπόλοιπη θεωρία των διαλέξεων	28
9. Πρακτική εξάσκηση με DevTools	31
10. Mock exams και πιθανές μικρές επεκτάσεις	35
11. Τελευταία επανάληψη και αναφορές	37

MovieLens: προετοιμασία προφορικής εξέτασης

Ελληνικές εξηγήσεις με English technical terms. Βασίζεται στις τέσσερις διαλέξεις του φακέλου `course_material`, στις εκφωνήσεις Άνοιξης και Ιουνίου 2026 και στον ελεγμένο κώδικα της εφαρμογής. Έκδοση: 10 Σεπτεμβρίου 2026.

Πώς να διαβάσεις

Ξεκίνα με αρχιτεκτονική, HTTP, JavaScript και FastAPI. Συνέχισε με HTML/CSS/DOM, SQLite και recommendations. Ολοκλήρωσε με την υπόλοιπη θεωρία, τις ασκήσεις DevTools, τα mock exams και το cheatsheet.

Για κάθε ερώτηση: απάντησε δυνατά χωρίς να διαβάζεις, εντόπισε το αντίστοιχο σημείο κώδικα και δείξε μία παρατήρηση στον browser. Μια καλή απάντηση έχει ορισμό, παράδειγμα από την εφαρμογή και μία σημαντική εξαίρεση. Οι ερωτήσεις είναι υλικό εξάσκησης, όχι εγγύηση για το περιεχόμενο της επόμενης εξέτασης.

Τι γνωρίζουμε για την εξέταση

Ο φοιτητής αναφέρει ότι στην προηγούμενη εξέταση ζητήθηκαν ερωτήσεις όπως «τι επιστρέφει `async`», «τι είναι `BaseModel`», λειτουργία API, `Inspect Element` και `Network requests`. Αυτή είναι αναφορά εμπειρίας, όχι διαθέσιμο επίσημο πρακτικό. Η εκφώνηση Ιουνίου είναι πραγματικό προηγούμενο πρακτικό θέμα: POST αναζήτηση tags με κανόνα μήκους 5 χαρακτήρων. Όλες οι άλλες προφορικές ερωτήσεις και οι προτεινόμενες επεκτάσεις εδώ είναι προβλέψεις από την ύλη. Δεν υπάρχουν δεδομένα για στατιστικές συχνότητες θεμάτων.

Κλειδί πηγών

Οι αριθμοί είναι οι φυσικές σελίδες του PDF ή η σειρά διαφάνειας του PPTX, με αρίθμηση από 1.

Κωδικός	Αρχείο	Έκταση	Ρόλος
I	1_Introductory_Lecture - v.2.1.pptx	69 slides	Εισαγωγή, αρχιτεκτονική, τεχνολογίες
H	2_WebDev_HTTP.pdf	77 σελίδες	HTTP και δίκτυα
JS	3_WebDev_Javascript.pdf	108 σελίδες	JavaScript και events
B	Server-side-programming-REST-Service s.pdf	129 σελίδες	Backend, REST, SQLite, fetch
S	WebApp_Dev_Assignment_Spring_2026.pdf	4 σελίδες	Βασική εργασία
J	WebApp_Dev_exams_assignment_2026_06.pdf	2 σελίδες	Επέκταση Ιουνίου

Το παλιό `Theory-to-Code-Correlation.pdf` (17 σελίδες), το αντίστοιχο Markdown και το `notes/study-notes.md` είναι προϋπάρχοντα βοηθήματα, όχι πρωτογενείς διαλέξεις. Διατηρήθηκαν ως έχουν και μπορεί να περιγράψουν την παλιά υλοποίηση. Για τον σημερινό κώδικα

χρησιμοποίησε αυτόν τον οδηγό. Το 20012.zip είναι προηγούμενο παραδοτέο, όχι πρόσθετη θεωρητική πηγή.

Κάλυψη πηγών

- I 7-30: Web, HTML/CSS/JS, client/server. I 31-52: servers, frameworks, XML/JSON, SOA/SOAP/REST. I 53-64: ασφάλεια, layers, virtualization, cloud/edge. I 68-69: Web evolution. Οι διοικητικές/εισαγωγικές διαφάνειες 1-6 και 65-67 δεν προσθέτουν τεχνικές ερωτήσεις.
- H 3-23: δίκτυα, WWW, servers. H 25-49: μηνύματα, μέθοδοι, URLs, headers, MIME. H 51-68: cookies, CORS, caching, chunking. H 69-76: HTTP/2 και εργαλεία.
- JS 2-24: runtime/event loop. JS 25-51: types, scope, equality, hoisting, loops. JS 52-95: objects, functions, closures, this, prototypes/classes. JS 97-107: DOM/events και ασκήσεις. Οι επαναλαμβανόμενες διαφάνειες animation ενοποιήθηκαν.
- B 4-18: client/server και περιεχόμενο. B 20-55: Servlets/JSP/Tomcat, cookies/sessions, JDBC, MVC. B 57-83: formats, SOAP, REST. B 85-95: FastAPI/Uvicorn/Nginx. B 98-114: SQLite. B 117-125: API clients, fetch, Promises.
- S 1-4: όλοι οι περιορισμοί και απαιτούμενες ροές. J 1-2: endpoint, matching, UI και σήμανση επέκτασης. Τα διαγράμματα και επιλεγμένα παραδείγματα κώδικα σε εικόνες ελέγχθηκαν οπτικά, όχι μόνο με εξαγωγή κειμένου.

Τα PDF διαλέξεων έχουν αναγνώσιμο text layer. Μεταδεδομένα: Η και JS δημιουργήθηκαν από Google, B από PowerPoint 2010, S/J από LaTeX. Μετρήθηκαν περίπου H 3.226, JS 4.792, B 5.970, S 589 και J 343 λέξεις στο text layer· εικόνες κώδικα δεν περιλαμβάνονται σε αυτούς τους αριθμούς. Τα slides χρησιμοποιούν κυρίως ενσωματωμένες γραμματοσειρές. Δεν εντοπίστηκαν μη αναγνώσιμα σημεία στις σελίδες που χρησιμοποιούνται ως τεκμήρια.

Τι ακριβώς υλοποιεί η εφαρμογή

Κίνηση χρήστη	HTTP endpoint	Backend	Αποτέλεσμα
Άνοιγμα σελίδας	GET / στη θύρα 8080	Python static server	HTML, μετά CSS και JS
Search	GET /movielens/api/movies?search=...	routes/movies.py: search_movies	JSON movies και πίνακας
Show average	GET /movielens/api/ratings/{id}	routes/movies.py: get_ratings	JSON ratings, μέσος όρος στον browser
Add	POST /movielens/api/movies	MovieAdd, add_movie	INSERT, commit, HTTP 201 και movieId
Submit/Remove rating	Κανένα request	Κανένα endpoint	Μεταβολή myRatings στη μνήμη
Get recommendations	POST /movielens/api/recommendations	RecommendationRequest, recommend	Υπολογισμός χωρίς αποθήκευση input
Search tags	POST /movielens/api/tags/movies	TagMoviesRequest, get_movies_for_tag	JOIN, GROUP BY, matchingTag

Ο API server ακούει στη θύρα 3000. Οι συναρτήσεις των routes είναι κανονικά def επειδή χρησιμοποιούν το σύγχρονο sqlite3. Ο browser κάνει asynchronous fetch. Η εφαρμογή έχει λογικό διαχωρισμό presentation, application και data layer, αλλά το SQLite εκτελείται μέσα στη διεργασία backend, όχι σε ξεχωριστό database server.

1. Αρχιτεκτονική και απαιτήσεις

Πηγές: I 7-30, 31-38, 46-58 · B 4-18, 54-59 · S 1-4. Υψηλή προτεραιότητα εξάσκησης λόγω άμεσης σύνδεσης με την επίδειξη της εργασίας.

Q001. Περιέγραψε την εφαρμογή σου σε ένα λεπτό

«Έφτιαξα ένα MovieLens explorer. Ο browser φορτώνει ένα HTML, ένα CSS και ένα vanilla JavaScript αρχείο από τη θύρα 8080. Το JavaScript στέλνει HTTP requests στο FastAPI στη θύρα 3000. Τα endpoints διαβάζουν ή γράφουν SQLite και επιστρέφουν JSON. Οι προσωπικές βαθμολογίες μένουν μόνο στη μνήμη της σελίδας και στέλνονται για τον υπολογισμό recommendations. Η αναζήτηση tags είναι η διακριτή επέκταση του Ιουνίου.» Δείξε frontend/index.js, backend/src/main.py και backend/src/routes/.

Q002. Ποια είναι η διαφορά Internet και WWW;

Το Internet είναι η δικτυακή υποδομή που συνδέει δίκτυα και μεταφέρει δεδομένα. Το WWW είναι σύστημα πόρων και υπερσυνδέσμων που λειτουργεί πάνω της, χρησιμοποιώντας URLs, HTTP και browsers. Email και άλλες υπηρεσίες χρησιμοποιούν επίσης Internet χωρίς να είναι κατ' ανάγκη Web. Η εφαρμογή μας είναι Web application επειδή προσπελάζεται με browser και HTTP. [I 7-8, H 3-16]

Q003. HTML, CSS, JavaScript και HTTP: τι κάνει το καθένα;

HTML: δομή και σημασιολογία περιεχομένου. CSS: παρουσίαση και layout. JavaScript: συμπεριφορά, events, τοπική κατάσταση και ενημέρωση DOM. HTTP: κανόνες επικοινωνίας request/response. Το HTTP μπορεί να μεταφέρει HTML, CSS, JS ή JSON· δεν είναι γλώσσα που σχεδιάζει τη σελίδα. Στο Network τα τρία αρχεία είναι ξεχωριστές αποκρίσεις με διαφορετικά Content-Type. [I 16-23, B 8, 15-17]

Q004. Ποιος κώδικας εκτελείται στον browser και ποιος στον server;

Το index.js εκτελείται στον browser. Τα Python modules εκτελούνται στον backend. Ο browser δεν εκτελεί SQL ούτε ανοίγει το movielens.db: καλεί endpoints. Στο Sources βλέπεις τον παραδομένο frontend κώδικα, όχι τον Python πηγαίο κώδικα. Το JSON είναι το αποτέλεσμα του server. Αυτό εξηγεί και γιατί δεν βάζουμε μυστικά σε JavaScript που κατεβάζει ο πελάτης. [I 24-28, B 4, 13-18]

Q005. Χρειάζονται διαφορετικοί υπολογιστές για client και server;

Όχι. Client/server περιγράφει ρόλους. Browser, static server και API μπορούν να τρέχουν στον ίδιο υπολογιστή, σε διαφορετικές διεργασίες και θύρες. Η διαφορά θύρας αρκεί για διαφορετικό origin. Το localhost αναφέρεται πάντοτε στον υπολογιστή που εκτελεί τον client, άρα σε άλλο laptop δεν δείχνει αυτόματα τον δικό μας server. [B 10-12, H 56]

Q006. Ποια είναι τα layers και τα tiers εδώ;

Presentation: HTML/CSS/JS. Application: routes και recommender. Data: SQLite schema και records. Είναι τρία λογικά layers. Δεν ισχυρίζομαι ότι υπάρχουν τρεις ανεξάρτητοι servers: το SQLite είναι embedded στο backend. Η διάκριση layer είναι λειτουργική, ενώ η φυσική κατανομή σε tiers αφορά deployment. [I 56-58, B 98]

Q007. Γιατί υπάρχουν APIRouter, models.py, db.py και recommender.py;

Τα routes αντιστοιχούν requests σε λειτουργίες. Τα Pydantic models περιγράφουν και ελέγχουν input. Το db.py ορίζει κοινά paths και το άνοιγμα/κλείσιμο σύνδεσης. Το recommender.py περιέχει τον αλγόριθμο. Ο διαχωρισμός επιτρέπει να αλλάξω τον αλγόριθμο χωρίς να αλλάξω το HTTP contract. Δεν είναι απαραίτητο να ονομάσω το σχήμα πλήρες MVC· έχει συγγενή διαχωρισμό ευθυνών. [B 54-55, 85-90]

Q008. Ποια δεδομένα επιβιώνουν από refresh ή restart;

Οι ταινίες που προσθέτω αποθηκεύονται στη βάση με commit και επιβιώνουν από refresh και restart. Οι dataset ratings και τα tags είναι επίσης στη βάση. Το myRatings, searchedMovies και το rendered αποτέλεσμα είναι page state: στο refresh ξαναδημιουργούνται. Το API δεν αποθηκεύει τις βαθμολογίες του recommendation request. Δεν χρησιμοποιούνται localStorage, sessionStorage ή cookies. [S 3-4]

Q009. Τι σημαίνει stateless και πώς συνυπάρχει με βάση δεδομένων;

Κάθε αίτημα περιέχει ό,τι χρειάζεται για την εξυπηρέτησή του χωρίς κρυφό conversational session στον server. Η μόνιμη κατάσταση των πόρων επιτρέπεται: π.χ. οι ταινίες στη βάση. Stateless δεν σημαίνει read-only, ούτε ότι ο server δεν έχει μνήμη. Στο recommendations στέλνω όλη τη λίστα ratings κάθε φορά· το «δεν αποθηκεύονται» είναι επιπλέον απαίτηση της εργασίας. [I 51, H 22-23, B 40, S 3]

Q010. Η σελίδα σου είναι static ή dynamic;

Τα τρία frontend αρχεία σερβίρονται στατικά, δηλαδή ο server στέλνει τα αποθηκευμένα αρχεία. Η συμπεριφορά της εφαρμογής είναι δυναμική: JavaScript μεταβάλλει το DOM από API responses χωρίς ολική επαναφόρτωση. Το backend παράγει JSON δυναμικά από queries και υπολογισμούς. «Static files» δεν σημαίνει «μη διαδραστική εφαρμογή». [I 16, 25-30, B 9, 14-17]

Q011. Γιατί vanilla JavaScript και όχι React;

Είναι ρητός περιορισμός της εκφώνησης: ακριβώς index.html, index.js, index.css χωρίς frameworks ή εξωτερικές frontend libraries. Τα απαιτούμενα forms, tables, events και fetch καλύπτονται από native browser APIs. Το ότι React αναφέρεται στις διαλέξεις δεν επιτρέπει να το χρησιμοποιήσω όταν η συγκεκριμένη εργασία το αποκλείει. [S 3-4, I 37]

Q012. Ποια είναι η ελάχιστη ροή εκκίνησης που πρέπει να εξηγήσεις;

Ενεργοποιώ το virtual environment με τις δηλωμένες dependencies και τρέχω ./start.sh both. Το Unicorn δημιουργεί την εφαρμογή, εκτελεί το lifespan startup και ακούει στη θύρα 3000. Αν λείπει η βάση, το setup την εισάγει από το bundled dataset. Ο static server σερβίρει το frontend στη θύρα 8080. Ανοίγω αυτή τη διεύθυνση στον browser. Το backend /docs είναι βοηθητικό interface ελέγχου. [B 85-93, S 1]

Q013. Ποιες απαιτήσεις καλύπτει η επέκταση Ιουνίου;

Προσθέτει textbox, button και πίνακα για tags, JSON body με search και POST /tags/movies. Για keyword μήκους κάτω από 5 απαιτεί ολόκληρο tag ίσο με το keyword· από 5 και πάνω συγκρίνει τους πρώτους 5 χαρακτήρες. Αγνοεί πεζά/κεφαλαία και εμφανίζει μία γραμμή ανά ταινία με matchingTag. Τα σχετικά blocks έχουν σχόλια με 10 παύλες στην αρχή και το τέλος. [J 1-2]

Q014. Τι δεν πρέπει να ισχυρισθείς ότι έχεις υλοποιήσει;

Δεν υπάρχουν login, accounts, sessions στον server, endpoint αποθήκευσης προσωπικών ratings, ORM, React, HTTPS setup, reverse proxy ή production deployment. Το /docs δημιουργείται από FastAPI. Η θύρα 8080 δεν σημαίνει ότι χρησιμοποιώ Tomcat: εδώ τρέχει python http.server. Η ειλικρινής αντιστοίχιση τεχνολογίας και κώδικα είναι σημαντικότερη από μια μεγάλη λίστα ονομάτων. [Κώδικας: start.sh, main.py, index.js]

Συνέχισε: [HTTP](#) · [FastAPI](#) · [DevTools](#).

2. HTTP, URLs, CORS και browser networking

Πηγές: Η 3-76 · Β 5-9, 40-45, 75-79. Για την εξέταση, περιέγραψε συγκεκριμένο request της εφαρμογής μαζί με τη θεωρία.

Q015. Πώς είναι δομημένο ένα HTTP request;

Στο HTTP/1.1 έχει request line, headers, κενή γραμμή και προαιρετικό body. Παράδειγμα: GET /movielens/api/movies?search=Toy HTTP/1.1, Host: localhost:3000. Το POST προσθήκης έχει Content-Type: application/json και body με title/genres. Τα DevTools παρουσιάζουν τα ίδια λογικά στοιχεία, ακόμη και όταν άλλη έκδοση HTTP έχει διαφορετική αναπαράσταση στο δίκτυο. [Η 25-43]

Q016. Πώς είναι δομημένο ένα HTTP response;

Στο HTTP/1.1 έχει status line όπως HTTP/1.1 201 Created, headers και προαιρετικό body. Εδώ το body της επιτυχούς προσθήκης είναι JSON με status και movielid. Τα response headers περιγράφουν το περιεχόμενο και πολιτικές όπως CORS. Το JSON πεδίο status δεν αντικαθιστά το πραγματικό HTTP status code. [Η 44-49, S 2]

Q017. Ανάλυσε ένα URL της εφαρμογής

Στο http://localhost:3000/movielens/api/movies?search=Toy%20Story το http είναι scheme, localhost host, 3000 port, /movielens/api/movies path και search=... query. Ένα #fragment θα χρησιμοποιούνταν από τον browser και δεν αποστέλλεται στο HTTP request target. Η διεύθυνση του frontend έχει άλλη θύρα, άρα άλλο origin. [Η 31-36, B 77]

Q018. URI, URL και URN είναι το ίδιο;

URI είναι γενικό αναγνωριστικό πόρου. URL είναι URI που εκφράζει τρόπο/θέση πρόσβασης, όπως ένα https URL. URN ονομάζει πόρο σε namespace, π.χ. urn:isbn:.... Δεν είναι σωστό να πεις «URI και URL είναι πάντα διαφορετικά»: το URL είναι ειδικότερη περίπτωση. [Η 32]

Q019. Γιατί encodeURIComponent στο search;

Κωδικοποιεί ένα συστατικό του URL ώστε χαρακτήρες όπως &, #, + ή ελληνικά να παραμείνουν μέρος της τιμής. Χωρίς encoding, το & θα μπορούσε να ξεκινήσει άλλο query parameter και το # να γίνει fragment. Δεν είναι encryption ούτε SQL escaping. Το backend κάνει ανεξάρτητα parameter binding για SQL. [Η 32-35 · index.js: searchMovies]

Q020. Τι κάνουν GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS;

GET διαβάζει representation. POST ζητά επεξεργασία δεδομένων, συχνά δημιουργία. PUT δημιουργεί ή αντικαθιστά την κατάσταση στο συγκεκριμένο target URI. PATCH εφαρμόζει μερική τροποποίηση. DELETE ζητά διαγραφή του πόρου. HEAD έχει σημασιολογία GET χωρίς response body. OPTIONS ζητά επιλογές επικοινωνίας και χρησιμοποιείται για CORS preflight. Η εφαρμογή ορίζει GET/POST· τα υπόλοιπα δεν είναι διαθέσιμα επειδή απλώς υπάρχουν στο HTTP. [Η 28-29, B 79]

Q021. Τι σημαίνει safe και τι idempotent;

Safe σημαίνει ότι ο client δεν ζητά αλλαγή κατάστασης του πόρου, όπως σε GET/HEAD. Idempotent σημαίνει ότι η επανάληψη του ίδιου αιτήματος έχει το ίδιο επιδιωκόμενο αποτέλεσμα στον πόρο με μία εκτέλεση. PUT/DELETE είναι idempotent ως προς τη σημασιολογία τους· δεν απαιτείται ίδιο response code κάθε φορά. POST /movies δεν είναι: δύο αποστολές μπορούν να δημιουργήσουν δύο records. Αυτός είναι ο λόγος να μην κάνω αυτόματο retry στην προσθήκη. [H 29 · εφαρμογή]

Q022. Γιατί το tag search είναι POST παρότι μόνο διαβάζει;

Το απαιτεί ρητά το προηγούμενο εξεταστικό θέμα. Η μέθοδος POST δεν σημαίνει υποχρεωτικό INSERT. Το endpoint δέχεται structured input και εκτελεί SELECT/JOIN. Η ίδια η εκφώνηση λέει ότι συνήθως η αναζήτηση θα υλοποιούνταν με GET. Αυτό είναι επιλογή του συγκεκριμένου contract. [J 1]

Q023. Τι διαφορά έχουν path, query και body parameters;

Path: /ratings/1, όπου το 1 ταυτοποιεί το ζητούμενο movie. Query: /movies?search=Toy, όπου η τιμή καθορίζει φίλτρο αναζήτησης. Body: POST /movies με JSON title/genres. Στο FastAPI το {movie_id} και η παράμετρος της συνάρτησης συνδέουν το path, μια απλή παράμετρος search εκτός path γίνεται query, και ένα BaseModel parameter δηλώνει body. [B 87-90]

Q024. Έχει ένα GET body; Είναι το POST πιο ασφαλές;

Το GET body δεν έχει γενικά καθορισμένη σημασιολογία και το browser fetch δεν επιτρέπει body για GET/HEAD. Στην εφαρμογή βάζουμε GET inputs στο URL. POST body δεν σημαίνει απόρρητο: στο απλό HTTP και URL και body είναι χωρίς κρυπτογράφηση. HTTPS προστατεύει τη μεταφορά. Το URL μπορεί επιπλέον να εμφανιστεί σε history/logs. [H 43 · συμπλήρωση: Fetch API]

Q025. Τι είναι MIME, Content-Type και Accept;

MIME/media type περιγράφει τον τύπο δεδομένων, π.χ. text/html, text/css, text/javascript, application/json. Content-Type λέει τι περιέχει το συγκεκριμένο body. Accept δηλώνει ποια response formats δέχεται ο client. JSON.stringify δημιουργεί JSON text, ενώ το Content-Type το δηλώνει ως JSON· το header μόνο του δεν μετατρέπει ένα object σε JSON. [H 38-41, B 6-8]

Q026. Ποιοι status codes έχουν σημασία εδώ;

200: επιτυχής ανάγνωση/υπολογισμός. 201: δημιουργήθηκε ταινία. 422: το request δεν περνά validation. 404: ανύπαρκτη διαδρομή. 405: η διαδρομή υπάρχει αλλά όχι για τη ζητούμενη μέθοδο, π.χ. GET /tags/movies. 500: απρόσμενο server error. Το /ratings/999999 επιστρέφει 200 και [] στη συγκεκριμένη υλοποίηση, όχι 404. 204 σημαίνει χωρίς body και δεν το διαβάζω με response.json(). [H 45, 49 · routes]

Q027. Γιατί μία σελίδα δημιουργεί πολλά requests;

Το αρχικό GET φέρνει HTML. Ο parser εντοπίζει link και script και ζητά CSS και JavaScript ξεχωριστά. Μετά οι ενέργειες χρήστη δημιουργούν fetch requests προς το API. Μπορεί να εμφανιστεί και favicon request. Ένα κλικ Submit rating δεν δημιουργεί request, ενώ ένα JSON POST μπορεί να προηγείται από OPTIONS. Δεν μετρώ μηχανικά «ένα κλικ ίσον ένα request». [B 14-17, H 61]

Q028. Τι είναι origin και Same-Origin Policy;

Origin είναι το tuple scheme, host, port. `http://localhost:8080` και `http://localhost:3000` έχουν διαφορετικό port. `localhost` και `127.0.0.1` είναι επίσης διαφορετικά host strings. Same-Origin Policy περιορίζει πώς script από ένα origin προσπελαύνει πόρους άλλου. Το CORS επιτρέπει επιλεγμένη διαμοίραση responses με HTTP headers. Δεν αρκεί να βρίσκονται οι εφαρμογές στον ίδιο υπολογιστή. [H 56-59]

Q029. Τι συμβαίνει σε CORS simple request;

Ένα cross-origin GET χωρίς μη επιτρεπόμενα custom headers συνήθως αποστέλλεται απευθείας. Ο server επιστρέφει `Access-Control-Allow-Origin` και ο browser ελέγχει αν το JS επιτρέπεται να διαβάσει το response. Απουσία preflight δεν σημαίνει απουσία CORS. Ακόμη και αν ο browser μπλοκάρει την ανάγνωση, ένα simple request μπορεί ήδη να έχει φτάσει στον server. [H 59-60]

Q030. Γιατί βλέπω OPTIONS πριν από POST;

Το `application/json` δεν είναι CORS-safelisted Content-Type. Ο browser πρώτα στέλνει preflight OPTIONS με `Origin`, `Access-Control-Request-Method: POST` και `Access-Control-Request-Headers: content-type`. Το middleware απαντά με επιτρεπόμενα origins/methods/headers. Αν ο έλεγχος πετύχει, ακολουθεί το πραγματικό POST. Δεν γράψαμε χειροκίνητο fetch για OPTIONS. Η άδεια μπορεί να είναι cached, οπότε OPTIONS δεν εμφανίζεται υποχρεωτικά κάθε φορά. [H 60-62 · `main.py`]

Q031. Το CORS προστατεύει τη βάση από μη εξουσιοδοτημένους χρήστες;

Όχι. Είναι browser policy για πρόσβαση από διαφορετικά origins, όχι authentication ούτε authorization. Ένα πρόγραμμα όπως `curl` δεν εφαρμόζει την πολιτική ανάγνωσης του browser. Η εργασία έχει δημόσιο `classroom API` χωρίς accounts. Σε πραγματικό deployment χρειάζονται ανεξάρτητοι έλεγχοι πρόσβασης· η αλλαγή `allow_origins` δεν τους αντικαθιστά. [H 56-62 · `main.py`]

Q032. Γιατί δεν λύνει το πρόβλημα το `mode: 'no-cors'`;

Επιτρέπει μόνο περιορισμένα requests και δίνει opaque response που το JavaScript δεν μπορεί να διαβάσει ως το επιθυμητό JSON. Δεν «απενεργοποιεί το CORS» ώστε να λειτουργήσει το UI. Η λύση είναι σωστό origin/URL και σωστά headers από τον server. Στο Network/Console ελέγχο preflight, τελικό response και τυχόν browser block. [H 56-62 · συμπλήρωση: Fetch API]

Q033. Γιατί λειτουργεί στο curl και αποτυγχάνει στον browser;

Ελέγχο πρώτα αν τα δύο εργαλεία καλούν ακριβώς το ίδιο URL/method/body. Αν ναι, πιθανή αιτία είναι CORS ή browser περιορισμός, επειδή `curl` δεν επιβάλλει Same-Origin Policy. Αν αποτυγχάνουν και τα δύο, εξετάζω αν τρέχει ο server, τη θύρα, το route και server logs. Το μήνυμα «Failed to fetch» μόνο του δεν αποδεικνύει ποια αιτία ισχύει. [H 56-62, 76]

Q034. Cookies και sessions: ποιος αποθηκεύει τι;

Το `Set-Cookie` είναι response header προς τον browser. Ο browser αποθηκεύει cookie και αποστέλλει κατάλληλα cookies με `Cookie` request header. Σε κλασικό server session το cookie συχνά κρατά μόνο session ID, ενώ τα δεδομένα βρίσκονται στον server. Cookie και session δεν είναι συνώνυμα. Το `myRatings` είναι απλό JS object στη μνήμη, όχι cookie ή server session. [H 53-54, B 41-45]

Q035. Τι σημαίνουν HttpOnly, Secure, SameSite, Max-Age;

HttpOnly αποκλείει ανάγνωση του cookie από JavaScript. Secure περιορίζει την αποστολή σε ασφαλή μεταφορά σύμφωνα με τους browser κανόνες. SameSite περιορίζει αποστολή σε ορισμένα cross-site contexts. Max-Age/Expires ορίζουν διάρκεια. Αυτά δεν κάνουν αυθαίρετα cookie περιεχόμενα έμπιστα. Η εφαρμογή δεν θέτει cookies· η ερώτηση αφορά συμπληρωματική θεωρία, όχι κρυμμένο feature. [B 43 · συμπλήρωση: browser cookies]

Q036. Cache-Control, ETag και 304: τι εξηγείς στο Network;

Caching επαναχρησιμοποιεί responses. Cache-Control δηλώνει πολιτική, π.χ. max-age χρόνο φρεσκάδας. ETag είναι validator συγκεκριμένης representation. Με If-None-Match ο client ρωτά αν άλλαξε· αν όχι, μπορεί να λάβει 304 και να χρησιμοποιήσει cached body. Last-Modified/If-Modified-Since βασίζονται σε χρόνο. no-cache επιτρέπει αποθήκευση αλλά απαιτεί revalidation· no-store ζητά να μην αποθηκευτεί. Δεν ισχυρίζομαι ότι τα custom API endpoints μας υλοποιούν ETag. [H 65-67]

Q037. Τι είναι Content-Length και chunked transfer;

Content-Length μετρά bytes του body, όχι χαρακτήρες ούτε αριθμό JSON πεδίων. Σε HTTP/1.1 chunked transfer στέλνει τμήματα με μήκος και τελικό zero-length chunk, όταν δεν είναι γνωστό εξαρχής το συνολικό μήκος. Δεν είναι το ίδιο με application pagination ούτε με JSON array. HTTP/2 και HTTP/3 έχουν δικό τους framing και δεν χρησιμοποιούν αυτόν τον μηχανισμό Transfer-Encoding: chunked. [H 49, 68]

Q038. HTTP/1.1, HTTP/2, HTTP/3: τι αλλάζει;

HTTP/1.1: text-based messages και persistent connections, με περιορισμούς ordering. HTTP/2: binary frames, πολλαπλά streams σε μία TCP σύνδεση και HPACK headers. Περιορίζει το HTTP-level head-of-line blocking αλλά παραμένει το TCP loss blocking. HTTP/3 χρησιμοποιεί QUIC πάνω από UDP και ανεξάρτητη παράδοση streams. Οι μέθοδοι/status codes κρατούν σημασιολογία. Το συγκεκριμένο Unicorn demo εξυπηρετεί HTTP/1.1. [H 30, 69-74]

Q039. Είναι το keep-alive ίδιο με user session;

Όχι. Keep-alive επαναχρησιμοποιεί δικτυακή σύνδεση. User session διατηρεί εφαρμογική σχέση ανάμεσα σε αιτήματα, συνήθως μέσω session ID. Μια εφαρμογή μπορεί να είναι stateless και να χρησιμοποιεί persistent TCP connections. Αντίστροφα, ένα session μπορεί να συνεχίζεται ενώ ανοίγουν νέες συνδέσεις. [H 22-23, 47, 74]

Q040. TCP, IP, DNS και ports: πού ανήκει το HTTP;

HTTP είναι application-layer protocol. IP αφορά διευθυνσιοδότηση/δρομολόγηση πακέτων. TCP δίνει αξιόπιστη, διατεταγμένη ροή bytes· ports διαχωρίζουν υπηρεσίες στο ίδιο host. DNS αντιστοιχίζει ονόματα σε δικτυακές πληροφορίες, συχνά IP addresses. Το διάγραμμα encapsulation βάζει application data μέσα σε TCP, IP και link frames. Δεν είναι κάθε HTTP request ένα TCP packet. [H 4-8, 14]

Συνέχισε: [JavaScript](#) · [DevTools](#).

3. JavaScript: async, τύποι, scope και functions

Πηγές: JS 2-95 · B 120-125. Το async/fetch είναι και θέμα που αναφέρθηκε από την προηγούμενη προφορική εξέταση.

Q041. Τι επιστρέφει μια async function στη JavaScript;

Πάντα Promise. Αν γράψω async function f() { return 5; }, το f() δίνει Promise που γίνεται fulfilled με 5. Το await f() δίνει την τιμή 5. Χωρίς return γίνεται fulfilled με undefined. Αν η συνάρτηση κάνει throw και δεν το χειριστεί, το Promise απορρίπτεται. Στο callApi, η τελική τιμή είναι το parsed JSON object, αλλά ο άμεσος caller λαμβάνει Promise. [B 122-124 · index.js: callApi]

Q042. Τι επιστρέφει μια Python async def;

Η κλήση μιας coroutine function, π.χ. async def f(): return 5, δημιουργεί coroutine object. Ο κώδικας αρχίζει όταν γίνει await ή προγραμματιστεί σε event loop. Αυτό διαφέρει από JavaScript Promise. Επιπλέον async def με yield είναι async generator function: το lifespan χρησιμοποιεί @asynccontextmanager για να μετατραπεί σε async context manager. Δεν ονομάζω όλα αυτά «Promise». [B 85, 92 · main.py: lifespan]

Q043. Τι ακριβώς κάνει await; Παγώνει ο browser;

Αναστέλλει τη συνέχεια της συγκεκριμένης async function μέχρι να settle η awaited τιμή/Promise. Με fulfillment συνεχίζει με την τιμή, με rejection ρίχνει την αιτία ως exception. Ο browser μπορεί να χειριστεί άλλα events όσο περιμένουμε network I/O. Ένας τεράστιος synchronous loop μέσα στην async function εξακολουθεί να μπλοκάρει το main thread· η λέξη async δεν τον μεταφέρει σε άλλο thread. [JS 3-24, B 122-124]

Q044. Τι είναι Promise και ποιες καταστάσεις έχει;

Είναι object για το αποτέλεσμα μίας asynchronous computation: pending, fulfilled ή rejected. Μετά το settlement δεν αλλάζει ξανά αποτέλεσμα. Το then προσθέτει callbacks και επιστρέφει νέο Promise. Το catch χειρίζεται rejection. Το finally εκτελεί cleanup ανεξάρτητα από επιτυχία/αποτυχία· ο callback δεν είναι υποχρεωτικό να επιστρέψει Promise. Ένα throw μέσα του μπορεί να κάνει reject το νέο Promise. [B 123-124]

Q045. Τι επιστρέφει fetch και γιατί έχουμε δύο await;

Το fetch επιστρέφει Promise που γίνεται fulfilled με Response όταν είναι διαθέσιμα status και headers. Το response.json() επιστρέφει άλλο Promise που διαβάζει το body και το κάνει parse. Έτσι στο callApi πρώτα περιμένουμε το Response και μετά το JSON. Response object και parsed data object είναι διαφορετικά πράγματα. Το body κανονικά διαβάζεται μία φορά, εκτός αν προηγηθεί clone. [B 122-125 · callApi]

Q046. Ένα HTTP 404 ή 422 μπαίνει αυτομάτως στο catch του fetch;

Όχι. Είναι έγκυρο HTTP response, οπότε το fetch συνήθως γίνεται fulfilled. Ελέγχω response.ok, που καλύπτει status 200-299, και κάνω throw για αποτυχία. Network/CORS failures απορρίπτουν το fetch. Αποτυχία JSON parsing επίσης μπορεί να κάνει throw. Η εφαρμογή διακρίνει HTTP status, μη έγκυρο JSON και αδυναμία επικοινωνίας. [B 124-125 · callApi]

Q047. Γιατί το `return response.json()` μπορεί να λειτουργήσει χωρίς δεύτερο `await`;

Μια `async function` που επιστρέφει `Promise` υιοθετεί την κατάσταση του. Άρα ο `caller` που κάνει `await` παίρνει τελικά `parsed JSON`. Όμως αν θέλω το τοπικό `try/catch` να πιάσει `rejection` κατά το `parsing`, πρέπει να το περιμένω μέσα στο `try`, π.χ. `return await response.json()`. Στο σημερινό `callApi` το `await` βρίσκεται ρητά στο `parsing block` για χρήσιμο `error message`. [Συμπλήρωση στην ύλη B 123-125]

Q048. Τι γίνεται αν ξεχάσω `await` στο `callApi`;

Το `data` είναι `Promise`, όχι `JSON object`. Πρόσβαση όπως `data.movies` δεν δίνει το `array` που περιμένω και ο επόμενος χειρισμός μπορεί να αποτύχει. Η σωστή ακολουθία είναι `const data = await callApi(...)`, έπειτα `data.movies`. Εναλλακτικά χρησιμοποιώ `.then(data => ...)`. Δεν διορθώνεται με τυχαίο `setTimeout`. [B 122-125]

Q049. Ποια είναι η σειρά εκτύπωσης σε αυτό το παράδειγμα;

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
// A, D, C, B
```

Ο `synchronous` κώδικας ολοκληρώνει πρώτα το τρέχον `task`. Η `Promise continuation` είναι `microtask` και εκτελείται πριν από το επόμενο `timer task`. Το `timeout 0` σημαίνει προγραμματισμό για αργότερα, όχι άμεση εκτέλεση. [JS 6-24, B 123 · `microtasks` ως συμπλήρωση]

Q050. Πώς συνεχίζει μια `async function` μετά από `await` μιας έτοιμης τιμής;

```
async function f() {
  console.log('A');
  await 5;
  console.log('B');
}
f();
console.log('C');
// A, C, B
```

Ακόμη και μια μη-`Promise` τιμή μπορεί να γίνει `await`. Η συνέχεια εκτελείται `asynchronous`. Ο κώδικας πριν από το πρώτο `await` τρέχει αμέσως στην κλήση της `JavaScript async function`. [Συμπλήρωση: `async semantics`]

Q051. Τι είναι `event loop`, `call stack` και `heap`;

`Call stack`: οι ενεργές κλήσεις συναρτήσεων. `Heap`: χώρος `objects`. Το `host` περιβάλλον διαχειρίζεται `network/timers` και προγραμματίζει `callbacks`. Το `event loop` οργανώνει την εκτέλεση `tasks` και `microtasks` όταν η προηγούμενη `synchronous` εργασία επιτρέπει συνέχεια. Το διάγραμμα των `slides` δείχνει `timer callback` στην `queue` όσο άλλες συναρτήσεις παραμένουν στο `stack`. [JS 4-24]

Q052. `Single-threaded` σημαίνει ότι δεν υπάρχουν `race conditions`;

Δεν εκτελούνται δύο `JS callbacks` ταυτόχρονα στο ίδιο `main thread`, αλλά υπάρχει `race` ως προς τη σειρά `asynchronous` αποτελεσμάτων. Μπορεί να αλλάξω `ratings` ενώ περιμένω `recommendations`. Το `ratingsVersion` καταγράφει ποια έκδοση χρησιμοποιήθηκε· αν αλλάξει πριν έρθει η απάντηση, η παλιά απάντηση αγνοείται. Αυτό είναι πρόβλημα χρονικής σειράς, όχι ταυτόχρονης εγγραφής από δύο `threads`. [JS 6 · `index.js: getRecommendations`]

Q053. var, let, const: ποιες είναι οι διαφορές;

var έχει function scope, επιτρέπει επαναδήλωση και αρχικοποιείται σε undefined πριν εκτελεστεί η assignment. let/const έχουν block scope και temporal dead zone πριν τη δήλωση. let επιτρέπει reassignment, const όχι. Στο const myRatings = {} επιτρέπεται myRatings[1] = ... γιατί μεταβάλλω το object, όχι το binding. Δεν επιτρέπεται myRatings = {}. [JS 28-31, 47-49]

Q054. Τι είναι hoisting και τι βγάζει το παράδειγμα foo των slides;

Hoisting είναι περιγραφή του ότι declarations λαμβάνονται υπόψη πριν από την εκτέλεση των statements· δεν μεταφέρονται πραγματικά γραμμές. Στο var foo=1; function bar(){ if(!foo){var foo=10;} alert(foo); } bar(); τυπώνεται 10. Η τοπική var foo σκιάζει την εξωτερική και αρχικά είναι undefined, άρα !foo είναι true. [JS 47-49]

Q055. Τι είναι temporal dead zone;

Το διάστημα από την είσοδο στο scope μέχρι την εκτέλεση της δήλωσης let/const, όπου το binding υπάρχει αλλά δεν επιτρέπεται πρόσβαση. console.log(x); let x=1; προκαλεί ReferenceError. Ακόμη και typeof x μέσα σε αυτό το διάστημα προκαλεί ReferenceError. Το typeof ενός πραγματικά αδήλωτου ονόματος επιστρέφει 'undefined'. [JS 28-31, 39, 47-49 · διευκρίνιση]

Q056. Ποιοι είναι οι primitive types;

string, number, bigint, boolean, undefined, symbol και null. Τα objects είναι μη primitive. typeof null επιστρέφει 'object' λόγω ιστορικής ιδιομορφίας, αλλά το null είναι primitive και διαφορετικό από undefined. Τα arrays είναι objects: Array.isArray([]) δίνει true, ενώ typeof [] δίνει 'object'. Το number περιλαμβάνει NaN και Infinity. [JS 26-27, 37-46]

Q057. Ποια είναι τα truthy και falsy;

Falsy: false, 0, -0, 0n, "", null, undefined και NaN, καθώς και το ειδικό legacy document.all. Άδεια arrays και objects είναι truthy. Το string '0' είναι truthy. Στο form η τιμή "" της placeholder option ελέγχεται ρητά. Δεν χρησιμοποιώ if(!value) για να αποφασίσω αν μια μεταβλητή έχει δηλωθεί. [JS 35-39]

Q058. == και ===: τι επιστρέφουν τα παραδείγματα;

'1200' == 1200 δίνει true λόγω coercion· '1200' === 1200 false. null == undefined true, null === undefined false. NaN === NaN false, γι' αυτό χρησιμοποιώ Number.isNaN. [] === [] false επειδή είναι διαφορετικά objects. Η strict equality δεν συγκρίνει αναδρομικά περιεχόμενα objects. [JS 34, 40-46, 52]

Q059. Γιατί κάνουμε Number(value) στα ratings;

Τα DOM input/select values είναι strings. Η εφαρμογή θέλει αριθμητικό rating και integer movielid στο JSON. Το Number('4.5') δίνει 4.5. Number("") δίνει 0, γι' αυτό ελέγχεται προηγουμένως το κενό. parseInt('4.5') δίνει 4 και θα έχανε το μισό αστέρι. parseFloat('4.5abc') δέχεται prefix ενώ Number δίνει NaN. [JS 35 · rateMovie/getRecommendations]

Q060. Τι κάνει JSON.stringify και τι JSON.parse;

stringify μετατρέπει JavaScript value σε JSON text, parse μετατρέπει έγκυρο JSON text σε JS value. JSON δεν υποστηρίζει functions, undefined, comments ή trailing commas. Στην εφαρμογή stringify φτιάχνει request body· response.json διαβάζει και κάνει parse το response. Δεν χρησιμοποιώ eval για να διαβάσω JSON. [I 44-45, B 62-63]

Q061. Είναι τα objects pass by reference;

Περνάμε τιμές· για ένα object η τιμή είναι αναφορά στο object, που μπορεί να μοιράζονται πολλά bindings. Με `const a=[1]; const b=a; b.push(2);` το `a` γίνεται `[1,2]`. Αν μια παράμετρος επαναδεθεί σε νέο object, δεν αλλάζει αυτόματα το binding του caller. Η μετάλλαξη κοινόχρηστου object και το reassignment διαφέρουν. [JS 26]

Q062. Τι κάνουν Object.keys, for...of και for...in;

`Object.keys` δίνει τα δικά του enumerable string keys ως array. Στο `myRatings` αυτά είναι movie IDs σε μορφή string. `for...of` διατρέχει τιμές iterable, όπως ένα array από movies. `for...in` διατρέχει enumerable property names, περιλαμβάνοντας ενδεχομένως inherited properties. Ένα απλό object δεν είναι εξ ορισμού iterable με `for...of`. [JS 51, 55, 94]

Q063. Τι σημαίνει ότι functions είναι first-class;

Μπορούν να αποθηκευτούν σε μεταβλητές, να περάσουν ως arguments και να επιστραφούν. Το `map` δέχεται callback και επιστρέφει νέο array. Το `sort` δέχεται comparator και μεταβάλλει το array. Ένας event handler είναι επίσης function value που καλείται όταν συμβεί event. Μια ordinary function χωρίς return επιστρέφει undefined. [JS 60-69]

Q064. Τι είναι closure και lexical scope;

Closure είναι συνάρτηση μαζί με πρόσβαση στο lexical environment στο οποίο δημιουργήθηκε. Δεν απαιτείται να αντιγράψει τις τιμές· κρατά πρόσβαση στα bindings. Παράδειγμα: `function makeAdder(x){return y => x+y;}` και `makeAdder(5)(2)` δίνει 7. Το `x` παραμένει προσβάσιμο μετά το return του `makeAdder`. Οι handlers της εφαρμογής προσπελούν τη shared κατάσταση του εξωτερικού script. [JS 73-79]

Q065. Τι τυπώνει ο timer loop με var και πώς διορθώνεται;

Στο `for(var i=0;i<5;i++){setTimeout(()=>console.log(i),1000);}` όλοι οι callbacks βλέπουν το ίδιο binding αφού ο loop ολοκληρωθεί: πέντε φορές 5. Με `let i` δημιουργείται binding ανά iteration και τυπώνεται 0,1,2,3,4. Στο δεύτερο quiz των slides, `function(i)` έχει δική του παράμετρο χωρίς να περάσουμε argument, άρα τυπώνει undefined. [JS 70-71]

Q066. Πώς καθορίζεται το this;

Για ordinary function εξαρτάται από τον τρόπο κλήσης: `obj.method()` έχει `this=obj`, `new F()` δίνει νέο instance, `call/apply/bind` καθορίζουν receiver. Σε standalone strict function το `this` είναι undefined. Arrow function δεν ορίζει δικό της `this`, το παίρνει lexical. Στο `onclick='showAverage(id, this)'` το `this` της inline handler είναι το button και περνά ως argument. [JS 56-59, 81, 86-87]

Q067. Τι διαφορά έχουν call, apply και bind;

`call` εκτελεί συνάρτηση με δοσμένο `this` και ξεχωριστά arguments. `apply` εκτελεί με `this` και array-like arguments. `bind` επιστρέφει νέα function με δεσμευμένο `this`/προκαθορισμένα arguments, χωρίς να την εκτελεί εκείνη τη στιγμή. Arrow `this` δεν αλλάζει με αυτά. Το `Person.call(this, ...)` των slides εκτελεί τον constructor ως ordinary function για αρχικοποίηση πεδίων, δεν συνδέει μόνο του όλο το prototype chain. [JS 81, 90]

Q068. Τι είναι prototype και class;

Objects μπορούν να αναζητούν properties κατά μήκος του prototype chain. Οι constructor functions έχουν συνήθως .prototype που γίνεται prototype των instances. Δεν έχουν όλα τα objects δική τους property με όνομα prototype. Η class syntax οργανώνει constructor/methods πάνω σε αυτό το μοντέλο. extends δημιουργεί σχέση κληρονομικότητας και super καλεί την υπερκλάση. Η εφαρμογή δεν χρειάζεται frontend classes. [JS 58, 89-93]

Q069. Υπάρχουν private πεδία στη σύγχρονη JavaScript;

Ναι, class private fields/methods με #, όπως #value. Οι closures μπορούν επίσης να κρύβουν κατάσταση μέσα σε lexical scope. Το παλιό slide που λέει «no standard way» πρέπει να διαβαστεί μαζί με το μεταγενέστερο slide που δείχνει #privateField. Δεν προσπαθώ να διαβάσω ένα private field έξω από το class. [JS 78, 93]

Q070. Τι είναι IIFE και singleton;

IIFE είναι function expression που καλείται αμέσως, π.χ. (()=>{...})(); Δημιουργεί δικό της scope. Το singleton παράδειγμα των slides κρατά ένα instance σε closure και επιστρέφει πάντα το ίδιο object από getInstance. Είναι θεωρητικά patterns, όχι απαραίτητα βήματα για κάθε μικρή εφαρμογή. Το δικό μας myRatings δεν είναι υλοποίηση singleton class. [JS 63, 79]

Q071. Τι επιστρέφει generator function;

Μια function* επιστρέφει Generator object. Το next() εκτελεί ως το επόμενο yield και δίνει object με value και done. Στο παράδειγμα generator(10), τα πρώτα next().value είναι 10 και 20· στο τέλος done είναι true. Generator δεν είναι Promise. Python generator, JS generator και async function είναι διαφορετικοί μηχανισμοί, παρότι μπορούν να σχετίζονται με αναστολή εκτέλεσης. [JS 64-65]

Q072. Ποια λάθη μπορούν να προκύψουν σε κλήσεις και callbacks;

Αν γράψω addEventListener('click', f()) καλώ τη f αμέσως και περνώ το αποτέλεσμα της· συνήθως θέλω f ή () => f(arg). Αν γράψω forEach(async ...) δεν περιμένει το forEach όλα τα async callbacks. Για σειριακές εργασίες προτιμώ for...of με await, για ανεξάρτητες εργασίες Promise.all. Η εφαρμογή έχει απλές κλήσεις με await και δεν χρειάζεται πρόσθετη concurrency βιβλιοθήκη. [JS 60, 102 · B 123]

Q073. Τι είναι optional chaining και γιατί υπάρχει στο ratingDropdown;

Το myRatings[movieId]?.rating επιστρέφει undefined αν δεν υπάρχει αντίστοιχη εγγραφή, αντί να πετάξει TypeError. Μετά το συγκρίνω με r ώστε προηγούμενη προσωπική βαθμολογία να εμφανιστεί selected. Δεν είναι μηχανισμός validation ούτε θεραπεία για κάθε λάθος· χρησιμοποιείται εκεί που η απουσία εγγραφής είναι φυσιολογική. [Κώδικας: ratingDropdown · συμπληρωματική σύνταξη]

Q074. Είναι η JavaScript απλώς interpreted και είναι ίδιο πράγμα με Java;

JavaScript και Java είναι διαφορετικές γλώσσες. Οι browser engines εκτελούν JavaScript με τεχνικές interpretation και συχνά JIT compilation. Το ECMAScript είναι η προδιαγραφή της γλώσσας· το DOM/fetch παρέχονται από το host. Το Node.js είναι άλλο host με διαφορετικά APIs. Δεν υπάρχει αυτόματα document στο Node. [I 19, JS 2-4, 25, 106]

Συνέχισε: [DOM και CSS](#) · [DevTools](#).

4. HTML, CSS, DOM και Inspect Element

Πηγές: I 16-23 · JS 97-107 · B 13-17, 120-121. Η λεπτομερής CSS και DevTools εξάσκηση είναι εφαρμογή των θεμάτων στην πραγματική εργασία, όχι ξεχωριστή διαθέσιμη διάλεξη CSS.

Q075. Τι είναι το DOM και πώς διαφέρει από τον HTML πηγαίο κώδικα;

Το DOM είναι η ζωντανή αναπαράσταση του εγγράφου ως δέντρο objects/nodes. Προκύπτει από parsing HTML αλλά αλλάζει από JavaScript. Το View Source δείχνει το αρχικό HTML, ενώ το Elements δείχνει το τρέχον DOM μαζί με τις γραμμές που πρόσθεσε το searchMovies. Αλλαγή στο Elements επηρεάζει την τρέχουσα σελίδα και συνήθως χάνεται στο reload. [JS 106, B 14-17]

Q076. Τι κάνει το DOCTYPE, το lang και το charset;

Το <!DOCTYPE html> ενεργοποιεί standards mode. Το lang='en' δηλώνει τη γλώσσα του αγγλικού UI για assistive tools. Το meta charset='UTF-8' καθορίζει encoding για σωστή απεικόνιση χαρακτήρων, π.χ. ελληνικών τίτλων. Δεν είναι HTTP status ή κάποια JavaScript ρύθμιση. Το viewport meta επιτρέπει το layout να προσαρμόζεται στο πλάτος κινητού.

Q077. id και class: πώς χρησιμοποιούνται;

Το id πρέπει να ταυτοποιεί μοναδικό element στο document. Η class μπορεί να χρησιμοποιείται σε πολλά στοιχεία. Στην εφαρμογή getElementById('search-results') εντοπίζει το tbody, ενώ οι .success και .error ορίζουν κοινά styles feedback. CSS #id επιλέγει ID και .class επιλέγει class. Η τιμή ενός id δεν δημιουργεί endpoint. [JS 102, B 16-17]

Q078. getElementById και querySelector: τι επιστρέφουν;

getElementById επιστρέφει το element με το ID ή null. querySelector δέχεται CSS selector και επιστρέφει το πρώτο matching element ή null. querySelectorAll δίνει NodeList. Αν αναζητήσω element πριν δημιουργηθεί ή με λάθος selector, πρόσβαση σε .value πάνω σε null προκαλεί σφάλμα. Στην εφαρμογή το script φορτώνεται μετά το HTML markup. [JS 98, 106 · index.html]

Q079. Γιατί το script βρίσκεται στο τέλος του body; Τι αλλάζει με defer/async;

Ένα ordinary εξωτερικό script εκτελείται όταν το συναντήσει ο parser και μπορεί να σταματήσει το parsing. Στο τέλος του body τα προηγούμενα στοιχεία υπάρχουν ήδη. Με defer στο head γίνεται παράλληλο download και εκτέλεση μετά το parsing, σε σειρά για deferred scripts. Με async η εκτέλεση γίνεται όταν είναι έτοιμο το script, χωρίς εγγυημένη σειρά. Η HTML attribute async και η JavaScript async function είναι διαφορετικές έννοιες. [JS 98, B 120]

Q080. Γιατί χρησιμοποιούμε form και preventDefault;

Το form προσφέρει φυσική υποβολή με Enter και built-in validation. Το submit event της εφαρμογής καλεί preventDefault ώστε να μη γίνει η προεπιλεγμένη navigation/submission. Μετά το addMovie/searchMovies κάνει fetch και ενημερώνει το ίδιο document. Χωρίς preventDefault μπορεί να γίνει reload και να χαθούν οι in-memory ratings. [JS 100-102 · index.html]

Q081. Γιατί required και label δεν αρκούν ως backend validation;

Το label συνδέεται με το input μέσω for/id, βοηθά screen readers και εστίαση. Το required δίνει άμεση καθοδήγηση στον browser. Ο client όμως μπορεί να αλλάξει HTML/JS ή να στείλει request με curl. Επομένως το backend ελέγχει ανεξάρτητα με Pydantic. Τα δύο validation layers έχουν διαφορετικό σκοπό: usability και όριο εμπιστοσύνης. [B 13, 90]

Q082. Ποια είναι η διαφορά value, textContent και innerHTML;

value διαβάζει την τρέχουσα τιμή input/select. textContent διαβάζει/γράφει κείμενο. innerHTML διαβάζει/γράφει serialized markup και κατά την ανάθεση το κάνει parse ως HTML. Αν ο τίτλος περιέχει `<b>Film</b>`, textContent τον εμφανίζει κυριολεκτικά ενώ raw innerHTML θα δημιουργούσε bold element. Η εφαρμογή χρησιμοποιεί escapeHtml για τα database strings μέσα στα table templates.

Q083. Πώς θα μπορούσε να προκύψει stored XSS;

Κακόβουλο markup θα μπορούσε να αποθηκευτεί ως movie title και να ερμηνευτεί ως HTML σε μεταγενέστερη αναζήτηση. Δεν αρκεί ότι το JSON είναι valid ούτε ότι το SQL είναι parameterized. Το escapeHtml χρησιμοποιεί textContent και παίρνει escaped HTML για το text context των κελιών. Αυτός ο helper δεν είναι γενικό sanitizer για URLs, CSS ή JavaScript attributes. Τα δυναμικά IDs στους handlers προέρχονται από integer database IDs.

Q084. Πώς ενημερώνεται ο πίνακας μετά το Search;

searchMovies διαβάζει input, κάνει encoded GET, περιμένει JSON, γεμίζει το searchedMovies map και κατασκευάζει rows. Τα title/genres γίνονται escaped text. Η ανάθεση tbody.innerHTML αντικαθιστά τις παλιές γραμμές. Δεν αλλάζει το HTML αρχείο στον δίσκο. Στο Elements βλέπεις νέους tr/td nodes, ενώ στο Network υπάρχει fetch JSON και όχι νέο document navigation.

Q085. Τι είναι event, callback και event handler;

Event είναι γνωστοποίηση ενέργειας/κατάστασης, π.χ. click, submit, change, keydown. Callback είναι function που περνάω για μεταγενέστερη κλήση. Event handler είναι callback συνδεδεμένο με συγκεκριμένο event. Η εφαρμογή χρησιμοποιεί inline handlers στο HTML. Η εναλλακτική addEventListener κρατά wiring στο JS και επιτρέπει πολλούς listeners. [JS 100-102, B 121]

Q086. Capturing, target, bubbling: ποια είναι η σειρά;

Το event κατεβαίνει από ancestors προς το target στη capturing phase, φτάνει στο target και, αν bubbles, ανεβαίνει στη bubbling phase. event.target είναι το στοιχείο όπου ξεκίνησε, currentTarget αυτό του listener που τρέχει. preventDefault ακυρώνει default action· stopPropagation εμποδίζει περαιτέρω propagation. Δεν είναι το ίδιο. Το διάγραμμα JS 104 δείχνει τις αντίθετες διαδρομές στο DOM tree.

Q087. Τι είναι event delegation και πότε θα το χρησιμοποιούσες;

Βάζω έναν listener σε ancestor όπως tbody και εξετάζω το target του event που ανεβαίνει. Έτσι χειρίζομαι και δυναμικές γραμμές χωρίς listener ανά button. Η τρέχουσα εφαρμογή χρησιμοποιεί απλά inline handlers ώστε να είναι άμεσα ορατή η αντιστοίχιση. Δεν ισχυρίζομαι ότι έχει ήδη delegation. Είναι πιθανή μικρή εξεταστική επέκταση του event bubbling. [JS 102-104]

Q088. Πώς εντοπίζεις το CSS που δίνει χρώμα σε ένα button;

Inspect στο button, μετά Styles: βρίσκω button και το background linear-gradient με var(--c1), var(--c2). Ακολουθώ τις custom properties στο :root. Στο Computed βλέπω τις τελικές τιμές. Μια διαγραμμένη declaration χάνει στο cascade ή έχει υπερκαλυφθεί· ελέγχω selector, specificity, source order, active pseudo-class και media query.

Q089. Πώς λειτουργεί το CSS cascade και η specificity;

Το cascade επιλύει ανταγωνιστικούς κανόνες λαμβάνοντας υπόψη προέλευση/importance, layers όπου υπάρχουν, specificity και σειρά. Μέσα στην ίδια κατηγορία, ID selectors έχουν μεγαλύτερο βάρος από classes/pseudo-classes και αυτές από element selectors. Σε ισοβαθμία συνήθως κερδίζει ο μεταγενέστερος κανόνας. Η inheritance μεταφέρει επιλεγμένες properties από parent· δεν μεταφέρονται όλες. Δεν αντιμετωπίζω το !important ως κανονικό εργαλείο για κάθε σύγκρουση.

Q090. Τι είναι το box model και το box-sizing: border-box;

Το box έχει content, padding, border και εξωτερικό margin. Με content-box το width αφορά κυρίως content· padding/border αυξάνουν το τελικό κουτί. Με border-box περιλαμβάνονται στο δηλωμένο width. Η εφαρμογή εφαρμόζει border-box σε όλα τα στοιχεία. Στο Computed/box model δείχνω πού βρίσκονται τα 10px padding ενός td και το border του.

Q091. display:none, visibility:hidden και opacity:0: διαφέρουν;

display:none αφαιρεί το box από το layout. visibility:hidden το κρύβει κρατώντας τον χώρο. opacity:0 το κάνει διαφανές, αλλά μπορεί να παραμένει διαδραστικό και να καταλαμβάνει χώρο. Δεν αρκεί οπτική απόκρυψη για να αλλάξω δικαιώματα ή να προστατέψω δεδομένα. [Σχετική βάση: B 17, dynamic style example]

Q092. Πώς προσαρμόζεται το UI σε μικρή οθόνη;

Το viewport meta ορίζει σωστή κλίμακα. Το @media (max-width:600px) μειώνει περιθώρια και βάζει τα form controls σε στήλη. Τα tables βρίσκονται σε wrapper με overflow-x:auto ώστε να γίνεται scroll μόνο εκεί. Το main document δεν πρέπει να βγαίνει οριζόντια εκτός οθόνης. Ελέγχω device toolbar σε 375px και δείχνω τον ενεργό media rule στο Styles.

Q093. Τι σημαίνουν :hover, :focus-visible, :disabled και nth-child;

Είναι pseudo-classes: επιλέγουν ανά κατάσταση/θέση. :hover αντιδρά σε pointer, :focus-visible δείχνει ορατό keyboard focus, :disabled δηλώνει μη ενεργό control, nth-child(even) χρωματίζει εναλλάξ γραμμές. Δεν απαιτείται JS για απλές αλλαγές εμφάνισης. Το button.disabled αλλάζει DOM property και επομένως την αντιστοίχιση του :disabled selector.

Q094. Ποια στοιχεία accessibility μπορείς να δείξεις;

lang, labels, semantic headings/main, πραγματικά button/form elements, th scope='col', labels για rating selects, visible keyboard focus και status messages με role='status'/aria-live. Τα table wrappers εστιάζονται με keyboard για scroll. Δεν βασίζομαι μόνο στο χρώμα error/success: το μήνυμα εξηγεί τι συνέβη. Δοκιμή μόνο με ποντίκι δεν αρκεί για keyboard λειτουργία.

Q095. Γιατί ένα button γίνεται disabled κατά το request;

Για αποφυγή επαναλαμβανόμενων clicks και διπλών requests όσο υπάρχει εκκρεμότητα. Το finally το επαναφέρει τόσο σε επιτυχία όσο και σε αποτυχία. Δεν δίνει απόλυτη server-side εγγύηση μοναδικής δημιουργίας, επειδή άλλος client μπορεί να στείλει POST. Η εφαρμογή κρατά επίσης το input σε failure και δεν ξαναστέλνει αυτόματα το POST.

Q096. Πώς διαφέρουν αλλαγή select, Submit, Remove και refresh;

Η αλλαγή select απλώς επιλέγει τιμή. Submit καλεί rateMovie, μετατρέπει σε number και γράφει myRatings. Remove κάνει delete το property και καθαρίζει αντίστοιχο select. Κανένα από αυτά δεν στέλνει rating στη βάση. Refresh ξαναφορτώνει το script και αδειάζει τη μνήμη. Στο Network πρέπει να μπορείς να αποδείξεις την απουσία rating request.

Συνέχισε: [FastAPI](#) · [DevTools](#).

5. FastAPI, Pydantic και ο κύκλος του request

Πηγές: B 85-95 · main.py, models.py, routes/*.py. Προτεραιότητα: BaseModel, routing, input sources και διαφορά server/framework/database.

Q097. Τι είναι FastAPI, Starlette, Pydantic και Unicorn;

FastAPI είναι το framework που δηλώνει endpoints και συνδέει types, validation και API documentation. Βασίζεται σε Starlette για ASGI/web υποδομή και Pydantic για data models/validation. Unicorn είναι ο server που δέχεται HTTP connections και καλεί το ASGI application. SQLite είναι ξεχωριστός ρόλος: αποθήκευση/queries. Το import fastapi δεν ξεκινά από μόνο του listening socket. [B 85, 92-93]

Q098. Τι είναι το BaseModel;

Είναι η βασική κλάση της Pydantic από την οποία κληρονομούν request models όπως MovieAdd. Οι annotated fields ορίζουν αναμενόμενη δομή και validation. Το FastAPI μετατρέπει το request JSON σε instance, ή επιστρέφει validation error. Δεν είναι database base class, δεν δημιουργεί SQL table και δεν αποθηκεύει κάτι αυτομάτως. Η δημιουργία row γίνεται μόνο από το INSERT του add_movie. [B 90]

Q099. Τι σημαίνει class MovieAdd(BaseModel) και title: str;

Το MovieAdd κληρονομεί συμπεριφορά από BaseModel. Το title: str είναι type annotation πεδίου. Η Pydantic το επεξεργάζεται μαζί με το Field(min_length=1). Η απουσία default σημαίνει required field. Το str_strip_whitespace αφαιρεί αρχικά/τελικά κενά πριν ελεγχθεί το μήκος. Άρα ένας τίτλος μόνο με κενά δεν γίνεται αποδεκτός. [B 90 · models.py]

Q100. Τι ακριβώς συμβαίνει όταν στέλνω JSON με λάθος δεδομένα;

Το FastAPI εντοπίζει το route, διαβάζει body και χρησιμοποιεί το model. Αν λείπει title ή ένα rating είναι 7, αποτυγχάνει validation πριν εκτελεστεί η συνάρτηση endpoint. Επιστρέφεται 422 με detail array που δείχνει loc, msg και type. Ο κοινός exception handler αφαιρεί το raw input από τα errors ώστε μη JSON αριθμοί όπως NaN να μη χαλούν και το error response. [models.py, main.py]

Q101. Ποιο validation υπάρχει για recommendations;

Κάθε RatingInput απαιτεί θετικό integer movieid και finite numeric rating από 0.5 έως 5, σε βήμα 0.5. strict=True δεν δέχεται numeric strings ή booleans για αυτά τα πεδία. Το RecommendationRequest απαιτεί list από τέτοια models και απορρίπτει διπλά movieid. Κενή λίστα επιτρέπεται και δίνει κενές recommendations. Ελέγχεται δομή/τιμή, όχι ύπαρξη κάθε ID στη βάση. [models.py]

Q102. Η Pydantic απορρίπτει πάντα επιπλέον πεδία;

Όχι. Το default extra policy είναι ignore. Τα τρέχοντα models δεν ορίζουν extra='forbid', επομένως ένα άγνωστο επιπλέον πεδίο αγνοείται. Αν απαιτούνταν strict schema ως προς τα ονόματα θα το δηλώναμε ρητά. Επίσης η Pydantic μπορεί να κάνει conversions όταν δεν ζητείται strict συμπεριφορά. Δεν συγχέω type annotation, coercion policy και extra-field policy. [B 90 · models.py]

Q103. Μια type annotation στην Python εκτελεί αυτόματα validation;

Όχι γενικά. Μια συνηθισμένη `def f(x: int)` δεν εγγυάται runtime type check. Οι annotations είναι metadata για εργαλεία και frameworks. Εδώ FastAPI/Pydantic τις χρησιμοποιούν ενεργά στο request boundary. Αν καλέσω ένα route function απευθείας από Python, παρακάμπτω τμήματα της HTTP διαδικασίας. Γι' αυτό οι backend έλεγχοι περνούν από το πραγματικό ASGI app. [B 88-90]

Q104. Τι κάνει ο decorator `@router.get('/movies')`;

Καταχωρεί συνάρτηση ως handler του συγκεκριμένου method/path. Δεν την καλεί για να τρέξει query κατά το import. Το `main.py` προσθέτει τον router με prefix `/movielens/api`. Το ίδιο `/movies` αντιστοιχεί σε άλλη συνάρτηση για POST. Μόνο το όνομα της Python function δεν ορίζει το URL. [B 86-89]

Q105. Τι κάνει το `APIRouter` και ποια η διαφορά του από το `app`;

Το `APIRouter` οργανώνει σχετικές routes ώστε το `main.py` να τις ενσωματώνει με `include_router`. Το `app` είναι το κύριο FastAPI application που καλεί ο server. Το prefix προστίθεται στις διαδρομές του router και δεν αλλάζει τα URL paths των static frontend αρχείων. Οι routers δεν είναι ανεξάρτητοι servers και δεν ακούνε σε δικές τους θύρες. [`main.py`, `routes`]

Q106. Γιατί τα endpoints έχουν `def` αντί `async def`;

Οι λειτουργίες `sqlite3` είναι blocking και δεν παρέχουν `await`. Το FastAPI εκτελεί normal `def` path operations σε thread pool. Αν τις γράφαμε `async def` και καλούσαμε το ίδιο blocking SQL μέσα τους, θα μπορούσαμε να μπλοκάρουμε το event loop. `async` δεν ισοδυναμεί με ταχύτερο CPU computation. Ο recommender επίσης κάνει Python υπολογισμούς. [B 85, 92 · FastAPI async documentation]

Q107. Τι είναι ASGI και πώς συνδέεται με HTTP;

ASGI είναι interface ανάμεσα σε Python application και server, με `scope`, `receive` και `send` για πληροφορίες και events. Unicorn μεταφράζει τη δικτυακή επικοινωνία σε αυτό το interface και το response πίσω σε HTTP. Δεν είναι request format που στέλνει ο browser. Το ότι ένα interface υποστηρίζει πολλούς τύπους επικοινωνίας δεν σημαίνει ότι ο συγκεκριμένος server/deployment ενεργοποιεί όλους τους HTTP versions. [B 92]

Q108. Τι είναι lifespan και τι κάνει `yield` εκεί;

Το `lifespan` ορίζει startup/shutdown context. Πριν το `yield` γίνεται `init_database`. Μετά το `yield` θα εκτελούνταν shutdown cleanup αν χρειαζόταν. Το `@asynccontextmanager` εφαρμόζεται σε `async generator` και δίνει `async context manager` που το FastAPI διαχειρίζεται. Δεν αποδίδουμε το `yield` ως «επιστροφή JSON στον browser». Το startup γίνεται πριν η εφαρμογή αρχίσει να εξυπηρετεί requests. [`main.py`]

Q109. Γιατί `if __name__ == '__main__':`;

Το block εκτελείται όταν τρέχω το module ως κύριο πρόγραμμα, π.χ. `python src/main.py`. Δεν εκτελείται όταν άλλο module κάνει `import main`. Έτσι tests ή `unicorn main:app` μπορούν να εισαγάγουν το application χωρίς να ξεκινήσει δεύτερος server. Το `main:app` σημαίνει module `main` και variable `app`. [B 93 · `main.py`]

Q110. Τι κάνει το CORS middleware στη διαδρομή του request;

Περιβάλλει το application και χειρίζεται preflight OPTIONS ή προσθέτει CORS headers σε κατάλληλα responses. Δεν χρειάζεται route /options ούτε manual SQL. Το τελικό URL εξακολουθεί να αντιστοιχίζεται στον ίδιο router. Στην εφαρμογή επιτρέπονται όλα τα origins για μη credentialed requests και οι μέθοδοι GET/POST. [H 58-62 · main.py]

Q111. Πώς ένα Python dict γίνεται JSON response;

Το FastAPI σειριοποιεί την επιστρεφόμενη δομή σε response με application/json. Τα sqlite3.Row μετατρέπονται με dict(row) για πεδία ονοματισμένα όπως movieId/title. Το Python None γίνεται JSON null, οι λίστες arrays. Δεν γράφουμε Python repr μέσα σε HTTP body: ένα Python dict με μονά εισαγωγικά δεν είναι από μόνο του JSON text. [B 62-63, 90-91]

Q112. Τι είναι OpenAPI, Swagger UI και response_model;

OpenAPI είναι περιγραφή του API: paths, methods, inputs και schemas. Το /openapi.json την εκθέτει ως JSON. Το /docs είναι Swagger UI που την παρουσιάζει και μπορεί να στείλει πραγματικά requests. response_model δηλώνει validation/filtering/τεκμηρίωση εξόδου. Τα σημερινά routes επιστρέφουν απλά dicts χωρίς typed response_model, άρα δεν πρέπει να ισχυριστώ ότι έχουν πλήρως τυποποιημένο output schema. [B 85, 90-91]

Q113. Τι διαφορά έχει request model από response model;

Request model ορίζει τι δέχεται ο server, response model τι εκπέμπει. Δεν χρειάζεται να είναι ίδιο: το POST /movies δέχεται title/genres αλλά επιστρέφει status/movieId. Στο παράδειγμα των slides UserIn περιέχει password ενώ UserOut όχι, ώστε να μη διαρρεύσει. Τα μοντέλα δεν είναι αυτομάτως permissions ή encryption. [B 91]

Q114. Πώς θα ακολουθούσες ένα POST /movies από άκρη σε άκρη;

submit event, preventDefault, addMovie, JSON.stringify, fetch με Content-Type, πιθανό OPTIONS, Unicorn, CORS middleware, POST route, MovieAdd validation, get_db, parameterized INSERT, commit, lastrowid, JSON 201, response.ok/json, feedback στο DOM. Μπορώ να σταματήσω σε κάθε βήμα και να πω τον τύπο των δεδομένων: strings input, JSON text, Python object, SQL parameters, rows/ID, JS object. [S 2 · κώδικας]

Q115. Τι απαντάς αν ένα request φτάνει σε λάθος endpoint;

Ελέγχω πλήρες URL, port, API_PREFIX, method και ακριβές path. GET /movies και POST /movies είναι διαφορετικές operations. /ratings/1 είναι path input, ενώ /ratings?movieId=1 δεν έχει δηλωθεί. Κλήση του API path στη θύρα 8080 πηγαίνει στον static server και πιθανότατα επιστρέφει HTML 404. Το να δω μόνο το όνομα της συνάρτησης δεν αρκεί. [B 86-89]

Q116. Τι χρειάζεται αλλαγή αν ο backend τρέξει αλλού;

Το API_BASE στο index.js πρέπει να δείχνει τον σωστό server και protocol/port. Η listening address 0.0.0.0 επιτρέπει interfaces, αλλά δεν είναι η διεύθυνση που μοιράζω ως host URL. Ελέγχω πρόσβαση δικτύου και CORS. Η εκφώνηση ζητά port 3000 και κοινό /movielens/api prefix· δεν αλλάζω αυτά χωρίς λόγο. [B 93, S 1]

Συνέχισε: [SQLite](#) · [recommendations](#).

6. SQLite, SQL και διαχείριση δεδομένων

Πηγές: B 98-114 · S 1 · db.py, setup_db.py και routes.

Q117. Τι είναι SQLite και πού βρίσκεται η βάση;

Embedded relational database engine. Δεν χρειάζεται ξεχωριστή υπηρεσία database server: η Python χρησιμοποιεί sqlite3 για να ανοίξει το backend/movielens.db. Το αρχείο είναι persistent storage. Οι διαδρομές υπολογίζονται από το `__file__` ώστε να μη δημιουργείται κατά λάθος άλλη βάση επειδή άλλαξε ο τρέχων φάκελος εκτέλεσης. [B 98, 102 · db.py]

Q118. Ποιο schema ζητά η εκφώνηση και πόσα δεδομένα περιέχει;

movies(movieId,title,genres), ratings(userId,movieId,rating,timestamp), tags(userId,movieId,tag,timestamp), με στήλες όπως στα CSV. Το bundled αρχείο εισάγει 9.742 movies, 100.836 ratings και 3.683 tag records. Το J αναφέρει 3.684 tags, αλλά οι πραγματικές data rows του bundled CSV είναι 3.683· header δεν είναι record. Μια υπάρχουσα βάση μπορεί να έχει πρόσθετες ταινίες. [S 1, J 1 · επαληθευμένο import]

Q119. Τι είναι primary key και composite primary key;

Primary key προσδιορίζει μοναδικά row. Στο movies, movieId INTEGER PRIMARY KEY είναι alias του SQLite rowid και όταν παραλείπεται στο INSERT η βάση επιλέγει ID. Στο ratings το σύνθετο key (userId,movieId) επιτρέπει μία βαθμολογία ανά dataset χρήστη/ταινία. Στο tags το (userId,movieId,tag) επιτρέπει διαφορετικά tags για την ίδια ταινία. Δεν μπερδεύω primary key με αριθμό γραμμής του CSV. [B 99 · setup_db.py]

Q120. Χρησιμοποιείς AUTOINCREMENT; Είναι αναγκαίο για unique IDs;

Όχι. INTEGER PRIMARY KEY αρκεί για μοναδικό ID μεταξύ υπαρχόντων rows. AUTOINCREMENT είναι ειδική επιλογή της SQLite που αλλάζει την πολιτική επαναχρησιμοποίησης IDs με πρόσθετο κόστος· δεν είναι συνώνυμο του primary key. Το cursor.lastrowid επιστρέφει το ID της δικής μας επιτυχούς εισαγωγής. Δεν υπολογίζουμε MAX(id)+1 στον client. [setup_db.py, add_movie]

Q121. Connection, cursor, execute, fetchone, fetchall: εξήγησέ τα

Connection εκπροσωπεί τη σύνδεση με τη βάση και διαχειρίζεται transactions. Cursor εκτελεί SQL και κρατά το αποτέλεσμα για ανάκτηση. conn.execute είναι συντόμευση που δημιουργεί/επιστρέφει cursor. fetchone δίνει επόμενη row ή None, fetchall όλες τις υπόλοιπες rows. Το row_factory=sqlite3.Row επιτρέπει πρόσβαση με όνομα στήλης και dict(row). [B 102-105]

Q122. Τι κάνει το with get_db() και τι το with conn;

To get_db είναι generator context manager που κάνει yield μια connection και την κλείνει σε finally. Δεν κάνει αυτόματο commit. Το native with conn διαχειρίζεται commit/rollback για transaction, αλλά δεν κλείνει τη connection. Στο setup χρησιμοποιούνται with conn και closing(conn) για δύο διαφορετικές ευθύνες. Αυτή η διαφορά διορθώνει παραπλανητικό σχόλιο των slides. [B 102, 107-109 · db.py/setup_db.py]

Q123. Τι είναι transaction, commit και rollback;

Transaction ομαδοποιεί database αλλαγές. Commit τις οριστικοποιεί, rollback ακυρώνει τις μη οριστικοποιημένες αλλαγές. Το add_movie κάνει commit μετά το INSERT. Το setup ξεκινά BEGIN πριν από DDL και inserts, ώστε να συμμετέχουν στο ίδιο transaction. Το απλό close δεν είναι commit: μη committed αλλαγές μπορεί να χαθούν. [B 104, 107]

Q124. Τι σημαίνουν ACID και atomicity στο setup;

Atomicity: όλες οι αλλαγές της συναλλαγής ή καμία. Consistency: διατήρηση των δηλωμένων κανόνων. Isolation: περιορισμός αλληλεπίδρασης ταυτόχρονων συναλλαγών. Durability: τα committed δεδομένα επιβιώνουν σύμφωνα με τις εγγυήσεις του συστήματος αποθήκευσης. Το setup επιπλέον χτίζει temporary DB και μόνο μετά την επιτυχία κάνει os.replace στο target. Έτσι αποτυχία εισαγωγής δεν αντικαθιστά τη λειτουργική βάση. [B 107 · setup_db.py]

Q125. Γιατί csv.DictReader και όχι split(',');

Οι τίτλοι συχνά περιέχουν κόμμα μέσα σε quoted πεδίο. Το csv module χειρίζεται quoting, escaped quotes και γραμμές σωστά. Το DictReader διαβάζει το header και δίνει dict ανά data row. Τα CSV values είναι strings, γι' αυτό μετατρέπουμε IDs/timestamps με int και ratings με float. Τα title/genres/tag μένουν strings. [setup_db.py]

Q126. Τι κάνει executemany; Είναι ένα τεράστιο SQL statement;

Εκτελεί το ίδιο parameterized statement για πολλές ακολουθίες parameters. Στο setup εισάγει τις λίστες από tuples με χαμηλότερο overhead από χειροκίνητο Python loop με πολλά commits. Δεν σημαίνει κατ' ανάγκη ένα μοναδικό INSERT statement για ολόκληρο τον πίνακα. Το transaction κρατά την εισαγωγή συνεπή και αποδοτική. [B 104]

Q127. Πώς αποτρέπεις SQL injection;

Τα SQL values δένονται με ? placeholders και ξεχωριστά parameters. Ένα title με εισαγωγικά αντιμετωπίζεται ως δεδομένο, όχι ως SQL σύνταξη. Στον recommender δημιουργούμε μόνο το πλήθος placeholders για IN (...), όχι τα ίδια τα IDs μέσα στο SQL string. Στο tags επιλέγεται ένα από δύο σταθερά WHERE fragments. Τα placeholders δεν χρησιμοποιούνται για table/column identifiers. [B 104, 111]

Q128. Γιατί το title search δεν χρησιμοποιεί απλώς LIKE;

Η εκφώνηση ζητά substring keyword. Το LIKE δίνει ειδική σημασία στα % και _, ενώ το instr(casefold(title), ?) τα χειρίζεται ως κανονικούς χαρακτήρες. Η Python str.casefold καταχωρείται ως SQLite function στο get_db και επιτρέπει case-insensitive Unicode σύγκριση. Η built-in SQLite LOWER/LIKE δεν προσφέρει πλήρες Unicode case folding χωρίς πρόσθετη υποστήριξη. Πρόκειται για συγκεκριμένη επιλογή της υλοποίησης.

Q129. Τι κάνουν JOIN, GROUP BY και MIN στο tag search;

JOIN συνδέει tags με movies μέσω moviend. Το WHERE κρατά μόνο matching tags. GROUP BY συγκεντρώνει τις γραμμές ανά ταινία και αποτρέπει επανάληψη movie με πολλά matching tags. MIN(t.tag) επιλέγει ένα πραγματικό tag από εκείνα που πέρασαν το WHERE· δεν υπολογίζει «καλύτερο semantic match». ORDER BY σταθεροποιεί την παρουσίαση. [routes/tags.py]

Q130. Dynamic typing σημαίνει ότι η SQLite αγνοεί πάντα τον τύπο της στήλης;

Όχι. Κάθε value έχει storage class NULL, INTEGER, REAL, TEXT ή BLOB, και οι στήλες συνήθως έχουν type affinity που μπορεί να μετατρέψει τιμές. Τα ordinary tables δεν έχουν strict typing όπως πολλοί άλλοι DBMS. INTEGER PRIMARY KEY έχει ειδική συμπεριφορά. SQLite υποστηρίζει και STRICT tables, αλλά δεν χρησιμοποιούνται εδώ. Δεν υπάρχει ξεχωριστή storage class Boolean ή datetime. [B 98-100, 112]

Q131. Έχεις foreign keys και indexes;

Το τρέχον schema έχει primary keys αλλά δεν δηλώνει foreign-key constraints μεταξύ ratings/tags και movies. Η σχέση εκφράζεται στα δεδομένα και στα JOINS· δεν ισχυρίζομαι ότι η βάση αποτρέπει κάθε orphan row. Τα primary keys προσφέρουν σχετική indexing/uniqueness υποστήριξη. Ένα ειδικό index σε ratings(movieid) θα μπορούσε να βοηθήσει σε μεγαλύτερη κλίμακα, αλλά θέλει μέτρηση και δεν είναι απαιτούμενο feature.

Q132. Γιατί μια SELECT δεν αλλάζει προσωπικά ratings; Πώς το αποδεικνύεις;

Η GET ratings διαβάζει dataset rows. Η POST recommendations εκτελεί μόνο SELECTs και υπολογισμό. Οι submitted ratings γίνονται list από tuples στη μνήμη της συγκεκριμένης κλήσης. Μπορώ να συγκρίνω counts ή snapshot της βάσης πριν/μετά. Οι αυτοματοποιημένοι έλεγχοι επιβεβαιώνουν ακριβώς ίδια bytes βάσης μετά από recommendations.

Q133. Πώς χειρίζεσαι setup, reset και αποτυχία import;

Κανονικό setup κρατά υπάρχουσα βάση και δεν εισάγει δεύτερη φορά τα ίδια primary keys. Explicit reset απαιτεί stopped backend και αντικαθιστά τα δεδομένα με το dataset, χάνοντας τις πρόσθετες ταινίες. Η αντικατάσταση γίνεται μόνο όταν ολοκληρωθεί το νέο import. Δεν τρέχω reset στην εξέταση για να διορθώσω κάθε σφάλμα· πρώτα διαβάζω logs και προστατεύω τα υπάρχοντα δεδομένα.

Συνέχισε: [αλγόριθμος](#) · [DevTools](#).

7. Recommendations: κατανόηση και αριθμητικό παράδειγμα

Πηγή αλγορίθμου: S 3 · υλοποίηση: backend/src/recommender.py. Οι επιλογές για μη ορισμένη Pearson, ties και missing ratings αναφέρονται ρητά για να μπορείς να τις υπερασπιστείς.

Q134. Ποιος είναι ο αλγόριθμος σε απλά λόγια;

User-based collaborative filtering. Συγκρίνω τις βαθμολογίες του προσωρινού χρήστη u με χρήστες v του dataset στα κοινά movies. Υπολογίζω Pearson similarity, κρατώ μέχρι $K=30$ χρήστες με χρήσιμο μη μηδενικό βάρος, και προβλέπω scores για movies που ο u δεν έχει βαθμολογήσει. Επιστρέφω μέχρι $N=10$ με την υψηλότερη πρόβλεψη. Δεν είναι content-based matching από genres/tags. [S 3]

Q135. Τι είναι co-rated items και γιατί δεν συγκρίνεις όλες τις θέσεις;

Είναι τα movies για τα οποία και ο u και ο v έχουν rating. Δεν μπορώ να θεωρήσω το missing rating ίσο με 0, διότι αυτό θα πρόσθετε ανύπαρκτη αρνητική προτίμηση. Φτιάχνω δύο vectors στην ίδια σειρά movie IDs. Ο χρήστης του browser δεν χρειάζεται νέο userId στη βάση· είναι το request-local input vector.

Q136. Γράψε και εξήγησε την Pearson correlation

Για C , το σύνολο κοινών movies, υπολογίζω μέσους μ_u_C και μ_v_C πάνω στο C . Έπειτα:

$$\text{sim}(u,v) = \frac{\sum((u_j - \mu_u_C) * (v_j - \mu_v_C))}{\sqrt{\sum((u_j - \mu_u_C)^2) * \sum((v_j - \mu_v_C)^2)}}$$

Οι sums είναι πάνω στα j που ανήκουν στο C . Το αποτέλεσμα είναι από -1 έως 1 , εκτός μικρών floating-point αποκλίσεων: $+1$ ίδια γραμμική τάση, -1 αντίθετη, κοντά στο 0 ασθενής γραμμική σχέση. Το magnitude δεν μετρά πόσα κοινά movies υπάρχουν.

Q137. Γιατί μία ή δύο ίδιες βαθμολογίες μπορεί να μη δίνουν αποτέλεσμα;

Για Pearson χρειαζόμαστε τουλάχιστον δύο κοινά items και μη μηδενική διακύμανση και στους δύο vectors. Με ένα item ή όλα τα ratings ίσα ο denominator μηδενίζεται. Η `_pearson` επιστρέφει sentinel 0 και αυτός ο neighbour δεν συνεισφέρει. Το UI ζητά δύο διαφορετικά scores, αλλά ακόμη και αυτό δεν εγγυάται επαρκή κοινά ratings με κάθε dataset user.

Q138. Ποιος μέσος χρησιμοποιείται στην πρόβλεψη;

Ο `mean_u` είναι ο μέσος όλων των υποβλημένων ratings του browser. Ο `mean_v` είναι ο μέσος όλων των dataset ratings του συγκεκριμένου neighbour. Αυτοί διαφέρουν από τους μέσους μόνο πάνω στα co-rated items που χρησιμοποιήθηκαν στην Pearson. Η mean-centering προσαρμόζει για χρήστες που βαθμολογούν γενικά αυστηρότερα ή επιεικέστερα.

Q139. Γράψε την εξίσωση πρόβλεψης και όρισε τον παρονομαστή

```
pred(u,i) = mean_u
            + sum(sim(u,v) * (rating(v,i) - mean_v))
            / sum(abs(sim(u,v)))
```

Και στις δύο sums συμμετέχουν μόνο οι επιλεγμένοι neighbours που έχουν rating για το candidate i . Αυτή είναι η ρητή ερμηνεία της υλοποίησης για missing ratings. Ο απόλυτος παρονομαστής αποφεύγει την ακύρωση θετικών και αρνητικών βαρών μεταξύ τους. Αν δεν υπάρχει usable weight, δεν δίνω αυθαίρετη πρόβλεψη. [5 3]

Q140. Γιατί κρατάς και αρνητικές correlations;

Η εξίσωση της εκφώνησης χρησιμοποιεί `sim` στον αριθμητή και `abs(sim)` στον παρονομαστή, άρα μπορεί να αξιοποιήσει αντίθετες προτιμήσεις. Αν ένας αντίθετος neighbour βαθμολόγησε ένα movie κάτω από τον δικό του μέσο, αυτό δίνει θετική συνεισφορά στον u . Η προηγούμενη positive-only επιλογή ήταν πρόσθετο φίλτρο. Τώρα ταξινομούμε μη μηδενικές similarities από μεγαλύτερη σε μικρότερη, με `userId` ως tie-breaker.

Q141. Δείξε ένα πλήρες παράδειγμα με αριθμούς

Ο u βαθμολογεί movie 1 με 5 και movie 2 με 1, άρα `mean_u=3`. Ο v_1 έχει ratings $[5,1,5,1]$ για movies $[1,2,3,4]$, `mean_v1=3` και `sim=+1` στα κοινά $[1,2]$. Ο v_2 έχει $[1,5,1,5]$ για $[1,2,3,5]$, `mean_v2=3` και `sim=-1`. Ο v_3 έχει $[4,2,5]$ για $[1,2,6]$, `mean_v3=11/3` και `sim=+1`.

Candidate	Αριθμητής	Παρονομαστής	Πρόβλεψη
movie 3	$1*(5-3) + (-1)*(1-3) = 4$	2	$3 + 4/2 = 5$
movie 6	$1*(5-11/3) = 4/3$	1	$3 + 4/3 = 4.33$
movie 4	$1*(1-3) = -2$	1	1
movie 5	$(-1)*(5-3) = -2$	1	1

Η σειρά είναι 3,6,4,5· στα ισόβαθμα scores μικρότερο movieId προηγείται. Τα ήδη rated 1 και 2 αποκλείονται. Αυτό το αποτέλεσμα ελέγχεται αριθμητικά στο `test_pearson_and_prediction_formula`, όχι μόνο οπτικά στον πίνακα.

Q142. Γιατί μπορεί ένα predictedRating να είναι πάνω από 5;

Ο mean-centered estimator δεν επιβάλλει μαθηματικά όρια $[0.5, 5]$. Με $\text{input } u=[5, 4]$, $\text{mean}_u=4.5$, οι ίδιες similarities του παραδείγματος δίνουν για *movie 3*: $4.5+2=6.5$. Η εφαρμογή κρατά raw scores για ranking και κάνει round σε δύο δεκαδικά στην έξοδο. Η κλίμακα 0.5-5 αφορά έγκυρες εισαγόμενες βαθμολογίες. Clamping θα ήταν πρόσθετη απόφαση, όχι το ίδιο ακριβώς output της εξίσωσης.

Q143. Γιατί δεν αποθηκεύεις τα submitted ratings;

Το απαιτεί η εκφώνηση. Η λίστα ratings χρησιμοποιείται στη συγκεκριμένη request. Η εφαρμογή δεν δημιουργεί νέο dataset user ούτε INSERT σε ratings. Statelessness από μόνη της δεν απαγορεύει writes· είναι διαφορετική έννοια. Ο αλγόριθμος δεν είναι pure function μόνο του body, επειδή το αποτέλεσμα εξαρτάται και από τη βάση που διαβάζει.

Q144. Τι συμβαίνει σε cold start, νέο movie ή κενό input;

Κενό input δίνει []. Με ένα rating ή ίδια scores μπορεί να μην υπάρχει usable Pearson. Νέο movie χωρίς dataset ratings δεν βοηθά την εύρεση neighbours και δεν ανακαλύπτεται ως candidate μέσω neighbourhood ratings. Άγνωστα IDs δεν αποθηκεύονται ούτε προκαλούν ειδικό existence error: δεν δίνουν overlap, αλλά η τιμή τους συμμετέχει στον mean_u . Είναι δηλωμένο όριο του απλού contract. Για πραγματικό προϊόν θα μπορούσε να ζητηθεί existence validation ή fallback, όχι σιωπηρή αλλαγή του ζητούμενου αλγορίθμου.

Q145. Γιατί K=30, N=10 και ποιο είναι το κόστος;

Η εκφώνηση επιτρέπει επιλογή K/N. Περιορίζουν γειτονιά και τελικό output. Ο κώδικας κάνει λίγα batched SQL lookups και Python loops πάνω στα σχετικά ratings, αντί ένα query ανά candidate. Είναι κατάλληλος για το small dataset· δεν ισχυρίζομαι ότι κλιμακώνεται αυτόματα σε εκατομμύρια χρήστες. Αρχικά θα μετρούσα query time/CPU και θα εξέταζα indexes ή preprocessing όταν χρειαστεί.

Q146. Ποια pitfalls πρέπει να ελέγξεις στον recommender;

Σωστή αντιστοίχιση movie IDs στα vectors, διαφορετικοί co-rated/global means, zero variance, abs στον παρονομαστή, αποκλεισμός ήδη rated movies, duplicate IDs, deterministic ties και ranking πριν από rounding. Δεν προσθέτω αυθαίρετο minimum support που αποκλείει candidates της εκφώνησης. Το sum of weights αφορά μόνο όσους έχουν το συγκεκριμένο candidate rating.

Συνέχισε: [υπόλοιπη θεωρία](#) · [DevTools](#).

8. Υπόλοιπη θεωρία των διαλέξεων

Πηγές: I 9-15, 31-64, 68-69 · B 20-83, 95. Αυτά τα θέματα υπάρχουν στην ύλη αλλά δεν είναι υλοποιημένα όλα στη MovieLens εφαρμογή. Η σύνδεσή τους είναι συγκριτική.

Q147. Τι είναι Java Servlet και ποιος το εκτελεί;

Server-side Java πρόγραμμα που εκτελείται σε servlet container, π.χ. Tomcat. Δέχεται requests και παράγει responses. Το HttpServlet παρέχει doGet/doPost κ.ά., ενώ οι request/response interfaces δίνουν πρόσβαση σε parameters, headers και output. Ο browser βλέπει το αποτέλεσμα, όχι εκτέλεση Java Servlet στον υπολογιστή του. Η αντίστοιχη ευθύνη στην εφαρμογή μας βρίσκεται στα FastAPI routes. [B 20-27]

Q148. Ποιος είναι ο κύκλος ζωής ενός Servlet;

init όταν φορτώνεται/αρχικοποιείται, service για κάθε request και destroy όταν αφαιρείται από λειτουργία. Το HttpServlet δρομολογεί από service σε doGet/doPost ανά μέθοδο. Αν εξυπηρετούνται concurrent requests, mutable instance fields μπορεί να είναι shared και απαιτούν προσοχή. Το lifespan της FastAPI έχει ανάλογο startup/shutdown ρόλο, αλλά δεν είναι Servlet. [B 22-24]

Q149. Τι είναι JSP και πώς γίνεται Servlet;

JSP είναι template με HTML και JSP elements. Ο container μεταφράζει το JSP σε Servlet, το κάνει compile/load και το εκτελεί. Με αλλαγή του JSP μπορεί να ξαναγίνει η translation. Η Java επεξεργάζεται στον server και η παραγόμενη HTML πηγαίνει στον browser. Στη δική μας εφαρμογή το HTML είναι static και το JS δημιουργεί το dynamic περιεχόμενο από JSON. [B 28-32]

Q150. Ποιες JSP scripting μορφές πρέπει να αναγνωρίζεις;

<% ... %> scriptlet με Java statements, <%= ... %> expression που γράφεται στην έξοδο, <%! ... %> declaration. Directives όπως <%@ page ... %> δίνουν metadata για τη μετάφραση. JSP actions είναι άλλη κατηγορία στοιχείων. Δεν μπερδεύω JSP syntax με JavaScript ούτε με JSON. Στην άσκηση αρκεί να εξηγήσω τη διαδικασία και ένα απλό παράδειγμα. [B 30-31]

Q151. Tomcat, WAR και WEB-INF: τι είναι;

Tomcat είναι Java servlet container/web server. WAR είναι archive της web application με συγκεκριμένη δομή. WEB-INF/classes περιλαμβάνει compiled classes και WEB-INF/lib JAR dependencies. Η deployment περιοχή του Tomcat είναι συνήθως webapps. Το frontend μας στη θύρα 8080 σεβρίρεται από Python, επομένως δεν έχει WAR ή Tomcat. [I 32, B 35-38]

Q152. Τι είναι JDBC και PreparedStatement;

JDBC είναι το Java API για σύνδεση με databases μέσω κατάλληλου driver. Connection, Statement/PreparedStatement και ResultSet αντιστοιχούν σε σύνδεση, εκτέλεση SQL και αποτέλεσμα. Το PreparedStatement δέχεται bound parameters για data values. Χρειάζεται κλείσιμο resources ή επιστροφή connection στο pool. Στην Python χρησιμοποιούμε sqlite3 με αντίστοιχη ιδέα, όχι JDBC driver. [B 48-52]

Q153. Model 1 και Model 2/MVC: πώς διαφέρουν;

Στο Model 1 το JSP λαμβάνει το request και συνδυάζει παρουσίαση με χρήση JavaBeans/data. Στο Model 2 ένα Servlet Controller χειρίζεται input και ροή, το Model διαχειρίζεται δεδομένα/λογική και το JSP View παρουσιάζει. Το διάγραμμα B 55 δείχνει αυτόν τον διαχωρισμό. Στη MovieLens υπάρχουν παρόμοιες ευθύνες σε JS UI, routes και recommender/DB, αλλά διαφορετική τεχνολογική υλοποίηση.

Q154. Τι είναι web service και API;

API είναι συμβόλαιο μέσω του οποίου λογισμικό ζητά λειτουργίες/δεδομένα. Web service εκθέτει λειτουργίες μέσω δικτύου με συμφωνημένα πρωτόκολλα και representations. Ο client μπορεί να είναι browser JS, command-line πρόγραμμα ή άλλο backend. Το contract της εργασίας περιγράφει method/path, inputs και JSON output· δεν απαιτεί ο client να γνωρίζει την εσωτερική SQL υλοποίηση. [I 46-50, B 57-59]

Q155. SOAP και REST: ποια είναι η ουσιαστική διαφορά;

SOAP είναι protocol μηνυμάτων, συνήθως με XML Envelope και ορισμένους κανόνες επεξεργασίας. REST είναι architectural style με constraints όπως statelessness και uniform interface. JSON δεν είναι υποχρεωτικό για REST· μπορεί να υπάρξει XML representation. SOAP συχνά χρησιμοποιεί HTTP ως transport, ενώ REST χρησιμοποιεί τη σημασιολογία resources/methods/statuses. Η MovieLens υλοποιεί HTTP/JSON API σύμφωνα με την εκφώνηση. [I 48-52, B 58, 67-79]

Q156. SOAP Envelope, Header, Body και Fault: τι περιέχουν;

Envelope περικλείει SOAP message. Το Header μπορεί να έχει metadata για processing, ενώ το Body το κυρίως περιεχόμενο. Fault περιγράφει σφάλμα SOAP processing. Το SOAP Header δεν είναι ίδιο πράγμα με τα HTTP headers που περιβάλλουν ολόκληρο το XML body. Στο slide B 68 φαίνεται ακριβώς αυτό: POST/HTTP headers και μέσα XML Envelope. [B 67-71]

Q157. Τι είναι WSDL και UDDI;

WSDL περιγράφει το service interface: types, operations/messages, bindings και network endpoints. UDDI αφορά registry για δημοσίευση και ανακάλυψη services. Η βασική ροή SOA είναι provider register, consumer find και bind/use. Στο σημερινό project το URL είναι γνωστό από το API_BASE και το OpenAPI περιγράφει το HTTP API· δεν υπάρχει UDDI registry. [I 46, B 69-73]

Q158. Ποιες είναι οι REST constraints;

Client-server, stateless, cacheable, uniform interface, layered system και προαιρετικό code-on-demand. Η uniform interface περιλαμβάνει αναγνώριση πόρων, διαχείριση μέσω representations, self-descriptive messages και hypermedia ως οδηγό κατάστασης. Η εργασία χρησιμοποιεί REST-style JSON endpoints, αλλά δεν υλοποιεί πλήρες hypermedia/HATEOAS. Δεν αποδεικνύεται πλήρης REST συμμόρφωση μόνο επειδή χρησιμοποιώ GET και POST. [I 51, B 75-79]

Q159. XML well-formed και valid: πώς διαφέρουν;

Well-formed σημαίνει συντακτικά σωστό XML: ένα root, σωστό nesting, matching tags, σωστό escaping. Valid σημαίνει επιπλέον συμμόρφωση με συγκεκριμένο DTD ή XML Schema. Ένα syntactically σωστό document μπορεί να έχει λάθος πεδίο/τύπο ως προς το schema. Είναι ανάλογη η διάκριση valid JSON syntax και αποδεκτού request σύμφωνα με Pydantic, χωρίς να είναι ίδια εργαλεία. [I 39-43, B 60-61]

Q160. XML Schema, JSON Schema και Pydantic: ποια η σχέση;

XML Schema/XSD περιγράφει δομή και constraints XML. JSON Schema περιγράφει δομή/constraints JSON. Pydantic είναι Python library που ορίζει models, κάνει runtime validation και μπορεί να παράγει JSON Schema. Το FastAPI ενσωματώνει αυτό το schema στο OpenAPI. Το schema είναι περιγραφή και ο validator είναι μηχανισμός ελέγχου· δεν είναι η ίδια έννοια. [B 60-63, 90]

Q161. YAML, JSON και XML: πώς τα συγκρίνεις;

JSON έχει objects/arrays και λίγους primitive types, με αυστηρή σύνταξη χωρίς comments. XML χρησιμοποιεί elements/attributes και namespaces, χρήσιμο για document structures και SOAP. YAML εκφράζει mappings/sequences συχνά με indentation και υποστηρίζει comments. Το --- είναι marker αρχής YAML document, όχι υποχρεωτική πρώτη γραμμή κάθε YAML file. Η εφαρμογή ανταλλάσσει JSON. [B 62-65]

Q162. Τι είναι Nginx και reverse proxy;

Nginx μπορεί να σερβίρει static files ή να δέχεται requests και να τα προωθεί σε backend όπως Unicorn. Reverse proxy βρίσκεται μπροστά από servers για κοινό entry point, routing ή TLS termination. Load balancer μοιράζει traffic. Αυτές οι λειτουργίες διαφέρουν από την business logic του recommender. Δεν υπάρχει Nginx configuration στο project. [B 95]

Q163. Authentication, authorization, confidentiality, integrity: εξήγησέ τα

Authentication: ποιος είσαι. Authorization: τι επιτρέπεται να κάνεις. Confidentiality: μη αποκάλυψη σε μη εξουσιοδοτημένους. Integrity: προστασία από μη επιτρεπτή αλλοίωση. Η εφαρμογή έχει validation και ασφαλή χειρισμό text/SQL, αλλά δεν έχει user login/authorization. CORS και Pydantic δεν υποκαθιστούν αυτούς τους ελέγχους. [I 53-55]

Q164. SSL/TLS και HTTPS: τι προστατεύουν;

HTTPS είναι HTTP πάνω από ασφαλή μεταφορά TLS, που προστατεύει επικοινωνία ως προς confidentiality/integrity και συνήθως πιστοποιεί τον server μέσω certificate. Τα slides χρησιμοποιούν τον παλιό όρο SSL· η σύγχρονη πρακτική είναι TLS. Δεν σημαίνει αυτόματα ότι ο χρήστης έχει γίνει authenticated στην εφαρμογή ή ότι το περιεχόμενο είναι ασφαλές από XSS/SQL injection. Το demo τρέχει σε HTTP localhost. [I 54-55]

Q165. Web 1.0, 2.0, 3.0: είναι versions του HTTP;

Όχι. Είναι όροι για τάσεις χρήσης/αρχιτεκτονικής του Web. Τα slides αντιπαραβάλλουν static/document consumption, συμμετοχικές/διαδραστικές υπηρεσίες και ιδέες semantic data, decentralization και AI. Δεν αποτελούν ακριβή protocol version negotiation όπως HTTP/1.1 ή HTTP/2. Η MovieLens είναι διαδραστική Web εφαρμογή· recommendations δεν την κάνουν από μόνα τους «Web 3.0». [I 9-15, 68-69]

Q166. Client-server και P2P: τι δείχνει το διάγραμμα;

Στο client-server οι clients απευθύνονται σε server που παρέχει υπηρεσία. Στο peer-to-peer οι κόμβοι μπορούν να έχουν και τους δύο ρόλους και να επικοινωνούν μεταξύ τους. Η εφαρμογή μας έχει κεντρικό API/database, άρα client-server. Δεν υπάρχει ανταλλαγή ratings μεταξύ browsers. [I 29]

Q167. Virtual machine, container και virtualenv: διαφέρουν;

VM συνήθως περιλαμβάνει guest OS πάνω σε hypervisor. Container απομονώνει διεργασίες με κοινό host kernel και πακετάρει εφαρμογή/dependencies. Python virtualenv απομονώνει Python packages/interpreter environment, όχι ολόκληρο OS ή kernel. Το backend/venv του project δεν είναι Docker container ούτε virtual machine. [I 62]

Q168. Τι είναι cloud, edge και η μεταξύ τους κατανομή;

Cloud παρέχει υπολογιστικούς πόρους ως υπηρεσίες με δυνατότητα κλιμάκωσης. Edge μεταφέρει μέρος της επεξεργασίας κοντύτερα στην πηγή δεδομένων/χρήστη, συχνά μειώνοντας latency ή δικτυακή κίνηση. Στο διάγραμμα I 64 sensors στέλνουν σε edge nodes και cloud για διαφορετική επεξεργασία. Αυτό δεν είναι μέρος του local MovieLens deployment. [I 63-64]

Διορθώσεις και προσεκτικές διατυπώσεις των slides

- JS 26/39: null είναι ξεχωριστή primitive τιμή, όχι undefined. Το typeof null παραμένει 'object'.
- JS 31: const απαγορεύει reassignment, όχι μεταβολή properties. JS 48: hoisting είναι μοντέλο κατανόησης, όχι κυριολεκτική μετακίνηση γραμμών.
- JS 54: eval δεν χρειάζεται για object literals/JSON και το εμφανιζόμενο m.model είναι 'Mustang', όχι 'Ford'. JS 67: constructor use χρειάζεται new για τα αναμενόμενα instances.
- JS 89: όχι όλα τα objects έχουν .prototype property· αναζήτηση γίνεται μέσω του εσωτερικού prototype link. Object.create(null) δεν έχει Object.prototype ancestor.
- H 43/49: η σημασιολογία body εξαρτάται από μέθοδο/status· το 201 μπορεί κανονικά να έχει body, όπως το δικό μας POST. Το 204 δεν έχει content.
- H 63: το browser Origin header είναι αυτό που χρησιμοποιεί το CORS, όχι inference από Referer. Τα δύο headers είναι διαφορετικά.
- B 102/108: with sqlite3.connect(...) δεν κλείνει από μόνο του connection. B 124: το .finally() επιστρέφει Promise, ο callback του δεν υποχρεούται να επιστρέψει Promise.
- Το HTTP/2 server push παρουσιάζεται ως δυνατότητα πρωτοκόλλου στα slides. Δεν το θεωρούμε εγγυημένη λειτουργία του σημερινού browser ή της εφαρμογής μας. Παλιές usage percentages/ημερομηνίες των slides δεν είναι σημερινά στατιστικά.

Οι παραπάνω διατυπώσεις διασταυρώνονται με την εκτέλεση του κώδικα και τις επίσημες συμπληρωματικές αναφορές του τελευταίου παραρτήματος. Στην εξέταση εξηγείς ήρεμα την ακριβή έννοια, όχι απλώς ένα σύνθημα.

Συνέχισε: [DevTools](#) · [mock exams](#).

9. Πρακτική εξάσκηση με DevTools

Πηγές: H 25-76 · JS 97-107 · B 117-125 · εφαρμογή. Οι ονομασίες παρακάτω ακολουθούν Chromium/Chrome DevTools· σε άλλον browser μπορεί να διαφέρουν λίγο. Πρόκειται για βήματα εξάσκησης και αναμενόμενα αποτελέσματα, όχι ισχυρισμό ότι κάθε panel δοκιμάστηκε αυτόματα.

Προετοιμασία επίδειξης

Εκκίνησε την εφαρμογή και άνοιξε <http://localhost:8080>. Άνοιξε DevTools με F12 ή Ctrl+Shift+I. Στο Network ενεργοποίησε recording και Preserve log, επέλεξε All αρχικά και καθάρισε τη λίστα. Για requests που ενδιαφέρουν το API χρησιμοποίησε μετά Fetch/XHR. Το Disable cache αφορά συνήθως το HTTP cache με ανοικτά DevTools· το preflight cache είναι ξεχωριστός μηχανισμός.

Να λες δυνατά τι σκοπεύεις να παρατηρήσεις πριν πατήσεις το κουμπί. Μη σβήνεις requests πριν τα εξηγήσεις. Για POST δημιουργίας χρησιμοποίησε μοναδικό δοκιμαστικό τίτλο ώστε να αναγνωρίσεις το record· κάθε πραγματική εκτέλεση δημιουργεί δεδομένα.

Άσκηση 1: Άνοιγμα σελίδας και αρχικά requests

Κάνε reload με Network ανοιχτό. Εντόπισε το document /, το index.css και το index.js στη θύρα 8080. Στο Headers δείξε Request URL, Request Method, Status Code και Content-Type. Στο Initiator εξήγησε ότι CSS/script προκύπτουν από HTML references. Τα API endpoints δεν καλούνται αυτόματα στην αρχική φόρτωση της συγκεκριμένης εφαρμογής. Μπορεί να υπάρχει favicon request που δεν ανήκει στις βασικές ροές.

Απάντηση εξάσκησης: «Η σελίδα δεν είναι ένα μοναδικό network αντικείμενο. Ο browser παίρνει το HTML, ακολουθεί link/script και στη συνέχεια το JavaScript μπορεί να ζητά JSON όταν ενεργώ.»

Άσκηση 2: Search και query parameter

Γράψε Toy Story και πάτησε Search. Φίλτραρε Fetch/XHR. Βρες GET προς `http://localhost:3000/movielens/api/movies?search=Toy%20Story`. Στο Payload/Query String Parameters δείξε τη decoded τιμή. Στο Response δες status:'success' και movies array. Στο Preview εμφανίζεται το ίδιο JSON σε δέντρο. Η συγκεκριμένη έκδοση dataset έχει τρία Toy Story titles.

Στο Elements άνοιξε tbody#search-results και ένα tr. Η API response δεν περιέχει HTML table· το searchMovies κατασκεύασε τα td. Στο Initiator ακολούθησε το index.js και βρες `callApi/searchMovies`. Το status:'success' μέσα στο JSON είναι διαφορετικό από HTTP 200.

Άσκηση 3: Show average

Στο Toy Story (movieId 1) πάτησε Show. Το GET /ratings/1 χρησιμοποιεί path parameter. Το response περιέχει individual dataset ratings, όχι έτοιμο average. Η showAverage υπολογίζει `sum/count` στον browser και εμφανίζει 3.92 (215) στο συγκεκριμένο dataset. Δείξε loop, division και toFixed(2). Για movie χωρίς ratings πρέπει να εμφανιστεί No dataset ratings, όχι διαίρεση με μηδέν.

Άσκηση 4: Submit rating χωρίς network request

Καθάρισε Network, διάλεξε 5 και πάτησε Submit στο Toy Story. Η γραμμή Your ratings εμφανίζεται και το count αυξάνεται, χωρίς request προς /ratings. Στο Console μπορείς να εξετάσεις myRatings και `Object.keys(myRatings)`. Είναι top-level lexical binding του κλασικού script: δεν χρειάζεται να είναι property του window.

Απάντηση εξάσκησης: «Η επιλογή select είναι string, το rateMovie την κάνει number και μεταβάλλει object στη μνήμη. Δεν εκτελείται database INSERT. Το GET ratings αφορά άλλα δεδομένα: τα ιστορικά ratings του dataset.»

Άσκηση 5: Recommendations και JSON body

Κράτησε Toy Story με 5 και αναζήτησε Twelve Monkeys. Βαθμολόγησέ το με 2. Πάτησε Get recommendations και βρες POST /recommendations. Στο Payload πρέπει να υπάρχουν numeric IDs 1 και 32 με ratings 5 και 2. Δείξε Content-Type: application/json. Η σειρά πεδίων στο JSON δεν αλλάζει το νόημα.

```
{ "ratings": [ { "movieId": 1, "rating": 5 }, { "movieId": 32, "rating": 2 } ] }
```

Στο Response υπάρχουν μέχρι 10 records με movieId/title/genres/predictedRating. Δεν θα περιλαμβάνονται τα ήδη rated IDs 1 και 32. Predicted scores μπορεί να είναι πάνω από 5 επειδή η

εξίσωση δεν κάνει clamp. Απάντησε ποιο path handler καλείται και πώς περνά τη λίστα στην recommend.

Άσκηση 6: Preflight και actual POST

Με Network σε All, αναζήτησε OPTIONS για το ίδιο endpoint και ξεχώρισέ το από POST. Στο preflight δείξε Origin: http://localhost:8080, Access-Control-Request-Method: POST και Access-Control-Request-Headers: content-type. Στο response δείξε Access-Control-Allow-Origin και Allow-Methods/Headers. Η CORS άδεια είναι έλεγχος browser πριν από το actual request.

Αν δεν εμφανίζεται OPTIONS, εξήγησε το preflight cache αντί να εφεύρεις αποτυχία. Νέο private browser context ή διαφορετικό endpoint μπορεί να δώσει νέο preflight. Το /docs στην ίδια origin του API δεν αναπαράγει το cross-origin περιβάλλον του frontend.

Άσκηση 7: Add movie και persistence

Πρόσθεσε π.χ. Exam Demo 2026 με genres Drama. Δείξε POST /movies, JSON body και HTTP 201 με νέο movieId. Αναζήτησε το ίδιο title, κάνε reload και αναζήτησέ το ξανά. Η ταινία παραμένει στη βάση ενώ τα προσωπικά ratings έχουν χαθεί. Στον κώδικα δείξε INSERT, commit και cursor.lastrowid. Μην ξαναστείλεις POST απλώς για να επαναλάβεις τη θέαση του response: δημιουργεί επιπλέον row.

Άσκηση 8: Backend validation με ελεγχόμενη αποτυχία

Άνοιξε http://localhost:3000/docs και POST /movies → Try it out. Στείλε title με μόνο κενά και genres Drama. Αναμένεις 422 με detail που εντοπίζει body.title, και καμία νέα εγγραφή. Εναλλακτικά από το Console του frontend:

```
const r = await fetch(API_BASE + '/recommendations', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({ratings: [{movieId: 1, rating: 7}]}))
});
console.log(r.status, r.ok, await r.json());
// 422, false, validation detail
```

Εξήγησε ότι το fetch fulfilled με Response· το HTTP failure δεν έκανε από μόνο του rejection. Τα DevTools επιτρέπουν να παρακάμψω το frontend, γι' αυτό είναι απαραίτητο το backend validation.

Άσκηση 9: June tags και exact/prefix rule

Γράψε funny και πάτησε Search tags. Η εφαρμογή πρέπει να κάνει POST /tags/movies με {"search":"funny"}, όχι GET. Στο dataset της δοκιμής βρέθηκαν 22 ταινίες. Δοκίμασε FUNNY και funny-extra: το πενταγράμματο prefix είναι το ίδιο. Με keyword κάτω από 5, π.χ. fun, χρειάζεται ολόκληρο tag ίσο με fun και όχι απλώς prefix του funny.

Δείξε το branch len(keyword)<5 στο tags.py, το substr(tag,1,5), το casefold, το bound ? και το GROUP BY. Για πολλά matching tags επιστρέφεται μία ταινία και ένα representative matchingTag. Τα αποτελέσματα άλλων keywords εξαρτώνται από τη βάση· μην απομνημονεύεις αυθαίρετα counts.

Άσκηση 10: Inspect Element, Styles και Computed

Inspect στο Search button. Δείξε τη δομή form/button και το submit handler. Στο Styles βρες background, padding και border-radius. Απενεργοποίησε προσωρινά μία declaration και παρατήρησε την αλλαγή. Στο Computed δείξε τελικές τιμές και box model. Βρες το :root και τις --c1/--c2 variables. Το UI change δεν τροποποιεί το αρχείο index.css στον δίσκο.

Στο device toolbar βάλε 375px. Βρες τον ενεργό media rule που αλλάζει flex-direction σε column. Με αποτελέσματα σε table, το οριζόντιο scroll πρέπει να περιορίζεται στον wrapper και όχι σε ολόκληρη τη σελίδα.

Άσκηση 11: Ασφαλές text και DOM mutation

Σε δοκιμαστική βάση πρόσθεσε title `<b>Exam</b>` και αναζήτησέ το. Πρέπει να φαίνεται το κυριολεκτικό text μαζί με τα tags, χωρίς bold element στο td. Στο Elements έλεγξε ότι δεν δημιουργήθηκε b node. Δείξε escapeHtml και εξήγησε γιατί το textContent προσφέρει ασφάλεια στο text context. Δεν χρειάζεται να εκτελέσεις κακόβουλο script για να αποδείξεις τη διαφορά.

Άσκηση 12: Breakpoint και τύποι δεδομένων

Στο Sources άνοιξε index.js και βάλε breakpoint στη γραμμή μετά το await callApi της searchMovies. Κάνε Search. Όταν σταματήσει, δείξε keyword ως string, data ως object και data.movies ως array. Χρησιμοποίησε Step over για τον loop και Resume για ολοκλήρωση. Με breakpoint πριν το fetch μπορείς να ακολουθήσεις call stack προς το event handler. Το await δεν σημαίνει ότι ο χρήστης παγώνει όλο τον browser μέχρι το δίκτυο.

Άσκηση 13: Offline ή request blocking

Μετά τη φόρτωση του frontend, από το Network επέλεξε προσωρινά Offline ή block το συγκεκριμένο API request. Κάνε Search. Πρέπει να εμφανιστεί error message και να ξεμπλοκάρει το button. Επανέφερε Online/unblock και δοκίμασε ξανά. Σύγκρινε με HTTP 422: στην offline περίπτωση δεν υπάρχει κανονικό server response με status 422. Μην αφήσεις το browser σε Offline πριν την επίδειξη.

Άσκηση 14: Slow network και αλλαγή state κατά την αναμονή

Ενεργοποίησε throttling, ζήτησε recommendations και αφαίρεσε ή άλλαξε rating πριν ολοκληρωθεί. Η παλιά response δεν πρέπει να εμφανιστεί ως recommendation για τη νέα λίστα. Δείξε ratingsVersion και σύγκριση version μετά το await. Το disabled rec button εμποδίζει δεύτερη παράλληλη υποβολή recommendations, ενώ ο version check καλύπτει αλλαγές input state.

Άσκηση 15: Network Timing και διάγνωση

Στο request Timing δείξε queueing/stalled, connection setup όπου υπάρχει, waiting/TTFB και content download. TTFB δεν είναι καθαρός χρόνος SQL: περιλαμβάνει δικτυακές και server αναμονές. Το waterfall δείχνει χρονική σχέση requests, όχι τη σειρά των Python functions. Στη localhost επίδειξη οι χρόνοι δεν αντιπροσωπεύουν production performance.

Άσκηση 16: Copy as cURL και αναπαραγωγή

Επέλεξε το GET search και Copy as cURL. Αναγνώρισε URL, headers και method. Για POST έλεγξε το body πριν το εκτελέσεις, επειδή replay του Add δημιουργεί νέο movie. Το curl είναι HTTP client και δεν δημιουργεί DOM ούτε εφαρμόζει browser CORS policy. Η αναπαραγωγή του ίδιου request βοηθά να ξεχωρίσεις frontend bug από backend συμπεριφορά.

Ελάχιστη αφήγηση 90 δευτερολέπτων

«Πατάω Search, το submit event ακυρώνει την κανονική φόρμα και καλεί JavaScript. Το keyword κωδικοποιείται στο query και το fetch δίνει Promise. Στο Network βλέπω GET, URL, 200 και JSON movies. Η FastAPI συνδέει τη διαδρομή με search_movies, που εκτελεί bound SQL και επιστρέφει dict. Με await διαβάζω Response και μετά JSON, ελέγχω status και γράφω ασφαλές text σε νέο tbody. Οι προσωπικές μου βαθμολογίες είναι ξεχωριστό object στη μνήμη και δεν αποθηκεύονται στον server.»

Επιστροφή: [ευρετήριο](#) · [mock exams](#).

10. Mock exams και πιθανές μικρές επεκτάσεις

Δεν υπάρχει πλήρης κατάλογος προηγούμενων προφορικών θεμάτων. Τα παρακάτω είναι προσομοιώσεις με βάση τις διαλέξεις και την αναφορά του φοιτητή. Η μόνη διαθέσιμη προηγούμενη επίσημη πρακτική εκφώνηση είναι η επέκταση tags του Ιουνίου 2026.

Mock A: βασική κατανόηση, 15 λεπτά

- 2 λεπτά: εξήγησε client/server/data flow και δείξε τους φακέλους.
- 3 λεπτά: Search, Network URL/query/response και DOM update.
- 3 λεπτά: τι επιστρέφουν async, fetch και response.json; γιατί 422 δεν απορρίπτει fetch;
- 3 λεπτά: BaseModel, fields, validation, query/path/body.
- 2 λεπτά: Show average έναντι τοπικού Submit rating.
- 2 λεπτά: εξήγησε refresh και persistence χωρίς να κοιτάς σημειώσεις.

Κριτήριο επιτυχίας: ο εξεταστής μπορεί να ακολουθήσει μία συγκεκριμένη ενέργεια από click μέχρι SQL και πίσω. Δεν αρκεί να επαναλάβεις ότι «καλώ το API».

Mock B: debugging και network, 20 λεπτά

- Δείξε OPTIONS/POST και εξήγησε το origin tuple.
- Στείλε invalid rating και εντόπισε το 422 body χωρίς να το πεις network failure.
- Εντόπισε τον πραγματικό CSS rule ενός button και το box model.
- Δείξε closure example, var/let timer loop και διαφορά arrow this.
- Εξήγησε SQL placeholders και γιατί ένα αποθηκευμένο title πρέπει να γίνει escaped.
- Κάνε offline request, επανάφερε Online και εξήγησε την αλυσίδα διάγνωσης.

Κριτήριο επιτυχίας: ξεχωρίζεις λάθος URL, μη διαθέσιμο server, CORS block, HTTP validation error και JS rendering error με συγκεκριμένα τεκμήρια.

Mock C: αλγόριθμος και ευρύτερη ύλη, 20 λεπτά

- Γράψε Pearson και εξήγησε co-rated means έναντι global means.
- Κάνε το αριθμητικό παράδειγμα του κεφαλαίου 7 χωρίς να διαβάξεις τη λύση.
- Εξήγησε cold start, zero variance και score πάνω από 5.

- Σύγκρινε Servlet/JSP/FastAPI, JDBC/sqlite3 και WSDL/OpenAPI.
- Δώσε REST constraints, cookies/session διαφορά και παράδειγμα caching.
- Εξήγησε πώς θα επέκτεινες ένα endpoint διατηρώντας το frontend contract.

Πιθανή επέκταση: φίλτρο genre

Πρόβλεψη εξάσκησης, όχι επίσημο μελλοντικό θέμα. Πρόσθεσε προαιρετικό query parameter genre στο GET /movies και ένα select στο frontend. Μελέτησε πρώτα τη μορφή pipe-separated genres ώστε Action να μη γίνει αυθαίρετο substring match άλλου token. Δέσε το value σε parameterized SQL και κράτησε το υπάρχον search. Έλεγξε συνδυασμό title/genre, κενό genre και απουσία αποτελεσμάτων. Μην προσθέσεις νέο framework.

Πιθανή επέκταση: ταξινόμηση αποτελεσμάτων

Ο client μπορεί να ταξινομήσει το ήδη ληφθέν array με sort και comparator για title. Αν ζητηθεί backend sorting, επιτρεπόμενα fields πρέπει να επιλεγούν από μικρό σταθερό mapping· ένα SQL identifier δεν μπορεί να δεθεί ως απλό ? value. Έλεγξε ties και αν η ταξινόμηση είναι numeric ή lexical. Πρόκειται για επέκταση, όχι ήδη διαθέσιμο control.

Πιθανή επέκταση: average στο backend

Μπορεί να ζητηθεί νέο endpoint με AVG(rating) και COUNT(*). Πρέπει να διατηρήσεις το ήδη απαιτούμενο GET /ratings/{id} που επιστρέφει όλες τις ratings, εκτός αν η νέα εκφώνηση το αλλάζει ρητά. Για μηδέν ratings το AVG δίνει NULL, όχι 0. Σχεδίασε σαφές JSON {average:null,count:0} και UI μήνυμα. Εξήγησε πού μεταφέρθηκε ο υπολογισμός και τι αλλάζει στο Network payload.

Πιθανή επέκταση: DELETE ή PATCH movie

Συμφώνησε πρώτα path/body/status. Έλεγξε existence, validation, parameterized SQL και συνέπειες για συνδεδεμένα ratings/tags. Η επιβεβαίωση διαγραφής στο UI δεν αντικαθιστά server authorization σε πραγματικό προϊόν. Μη δημιουργήσεις destructive endpoint απλώς επειδή υπάρχει στις διαλέξεις. Στην τρέχουσα εργασία δεν έχει υλοποιηθεί.

Πιθανή επέκταση: pagination

Αν ζητηθεί, όρισε offset/limit με bounds και σταθερό ORDER BY. Το contract πρέπει να εξηγεί αν επιστρέφονται count/next πληροφορίες. Έλεγξε κενή σελίδα και τελευταία σελίδα. Η υπάρχουσα spring απαίτηση ζητά όλα τα matching movies, άρα δεν προσθέτουμε σιωπηλό LIMIT που κρύβει αποτελέσματα. [S 2]

Μικρές ασκήσεις JavaScript από τη διάλεξη

- Counter με αρχική τιμή/step, increase/decrease/getValue, πρώτα με closure και μετά class. [JS 77-78, 92-93]
- Εξήγησε τα αποτελέσματα του quiz των τριών IIFEs: 12, 10, 15 αντίστοιχα, με lexical scope. [JS 95]
- Object με τρία αριθμητικά properties και μέθοδο sum, με literal/constructor/class. [JS 107]
- Δημιούργησε input click handler και link mouseover handler, έπειτα δείξε bubbling. [JS 105]

Αυτοαξιολόγηση

Βαθμολόγησε κάθε απάντηση 0-2: 0 δεν μπορώ να την εξηγήσω, 1 σωστός ορισμός χωρίς απόδειξη, 2 σωστός ορισμός και επίδειξη/παράδειγμα. Επανάλαβε μόνο τα 0/1. Στόχος είναι να απαντάς σε μεταβολή του παραδείγματος, όχι να απομνημονεύσεις ένα κείμενο. Η βαθμολογία αυτή είναι προσωπικό εργαλείο μελέτης, όχι πρόβλεψη βαθμού μαθήματος.

Επιστροφή: [ευρετήριο](#) · [cheatsheet](#).

11. Τελευταία επανάληψη και αναφορές

Οι απαντήσεις που πρέπει να βγαίνουν αμέσως

Ερώτηση	Απάντηση μίας πρότασης
JS async return;	Promise, του οποίου την τιμή παίρνω με await/then.
Python async def call;	Coroutine object· async def με yield δίνει async generator.
fetch return;	Promise που γίνεται fulfilled με Response, όχι απευθείας JSON.
response.json return;	Promise για διάβασμα/parsing του body.
HTTP 422 κάνει fetch reject;	Όχι, ελέγχω response.ok/status.
BaseModel;	Pydantic base class για validation/schema, όχι ORM/table.
Type hints μόνο;	Δεν εγγυώνται γενικό Python runtime validation.
Origin;	Scheme, host, port.
Γιατί OPTIONS;	Preflight για cross-origin JSON POST, πιθανόν cached.
CORS = authentication;	Όχι, είναι browser response-sharing policy.
To Submit rating κάνει POST;	Όχι, αλλάζει μόνο myRatings.
Μέσος όρος πού;	Στο browser από τις GET dataset ratings.
Recommendations πού;	Στον backend από request ratings και dataset.
Stateless = χωρίς DB;	Όχι, χωρίς remembered conversational client session.
SQL injection defense;	Bound parameters, όχι concatenation user values.
XSS defense εδώ;	Escape untrusted text πριν από table innerHTML.
with conn κλείνει DB;	Όχι· transaction management και close είναι διαφορετικά.
const object immutable;	Όχι· immutable binding, mutable contents.
Σε νέο reload;	Χάνεται page memory, παραμένει committed DB.
Pearson με ένα item;	Undefined· η εφαρμογή το θεωρεί μη usable signal.

Πού δείχνω στον κώδικα

Θέμα	Αρχείο και σύμβολο
async/HTTP/errors	frontend/index.js: callApi
POST JSON / 201 feedback	frontend/index.js: addMovie
query encoding / DOM	frontend/index.js: searchMovies
memory / number conversion	frontend/index.js: rateMovie, showMyRatings
average / path	frontend/index.js: showAverage
async stale result	frontend/index.js: getRecommendations, ratingsVersion
markup safety	frontend/index.js: escapeHtml
form / labels / tables	frontend/index.html
cascade / box model / mobile	frontend/index.css
CORS / lifecycle / routing	backend/src/main.py
BaseModel / Field / validators	backend/src/models.py
connection / caseload	backend/src/db.py: get_db
schema / import / transaction	backend/src/setup_db.py: initialize_db
GET / POST / INSERT	backend/src/routes/movies.py
request-local computation	backend/src/routes/recommendations.py
June POST / JOIN / prefix	backend/src/routes/tags.py
Pearson / weighted prediction	backend/src/recommender.py

Ελάχιστος έλεγχος πριν την εξέταση

- Ξεκινά η εφαρμογή από τις οδηγίες README, χωρίς internet για το bundled dataset.
- Κάνω Search, Show, Add, Submit/Remove, Recommendations και Search tags.
- Ξέρω να δείξω Request URL, method, query/body, status, response και initiator.
- Δείχνω DOM mutation, CSS rule και έναν breakpoint μετά από await.
- Ξεχωρίζω browser memory, persistent DB και request-local variables.
- Έχω επαναφέρει Online, αφαιρέσει breakpoints και κλείσει περιττά test tabs.
- Δεν χρησιμοποιώ το παλιό 20012.zip ως απόδειξη του σημερινού κώδικα: δεν ανανεώθηκε αυτόματα.

Πρωτογενείς online αναφορές για διευκρινίσεις

Οι τοπικές διαλέξεις είναι η κύρια βάση του οδηγού. Οι παρακάτω επίσημες αναφορές χρησιμοποιούνται για τις λεπτομέρειες που διορθώνουν/συμπληρώνουν σύντομες διατυπώσεις των slides. Έλεγχος αναφορών: 10/09/2026.

- MDN: [async function](#)
- MDN: [Using Fetch](#)
- MDN: [Promise.finally](#)
- MDN: [CORS](#)
- MDN: [Cookies](#)
- MDN: [const](#)
- MDN: [null](#)
- FastAPI: [async and def](#)
- Pydantic: [Models](#)
- Pydantic: [Fields](#)
- Python: [sqlite3](#)
- SQLite: [Datatypes and affinity](#)

Πώς επαληθεύτηκε ο οδηγός

Ελέγχθηκαν οι τέσσερις διαλέξεις και οι δύο εκφωνήσεις, με επιλεγμένα διαγράμματα και code-image slides οπτικά. Διασταυρώθηκαν η έννοια origin στο H 56, το BaseModel στο B 90 και το Pearson/weighted formula στο S 3. Η παγίδα με context-manager/connection ελέγχθηκε στα B 102/108 και στην επίσημη sqlite3 τεκμηρίωση. Το αριθμητικό παράδειγμα του κεφαλαίου 7 είναι executable regression check.

Το περιεχόμενο αντιστοιχεί στον σημερινό κώδικα. Το test script χρησιμοποιεί προσωρινή βάση. Η browser δοκιμή χρησιμοποιεί αντίγραφο της βάσης, όχι αλλαγή του υπάρχοντος dataset του φοιτητή. Οι χρόνοι επανάληψης/mock exams είναι προτάσεις μελέτης, όχι επίσημη διάρκεια προφορικής εξέτασης.

Επιστροφή: [ευρετήριο](#).